EE2211 Introduction to Machine Learning

Lecture 6

Shaojing Fan
fanshaojing@nus.edu.sg

Office hour (hybrid): Friday 10.30-11.30am, E4-05-26 (Weeks 5-10)

© Copyright EE, NUS. All Rights Reserved.

Help Us Improve: Mid-Course Survey

Your Feedback Matters! Welcome to Share Your Thoughts on EE2211 (Short Anonymous Survey)

https://docs.google.com/forms/d/e/1FAIpQLSerHWnUjU1IBfoo9fpemxuv1wib8WvY7tgm15NBZYO-T1jOxA/viewform?usp=header

© Copyright EE, NUS. All Rights Reserved.

We hear you, and we're working on it

- I would benefit from more real-life examples and demo code
- The main points of each lecture are a bit vague at times, can perhaps be displayed a little better.
- Ensure the examples are sufficient as some concepts may seem hard to understand at first.
- Can the lecture notes final version can be confirmed a bit earlier as I prefer hardcopy notes
- Coding tutorial
- Live Python coding demos during lecture to show how to solve the tutorial questions
- Connecting the lecture and tutorial

Challenges

Learning python code, it'll be better to have more codes to guide us and a list of essential code

I think finding the right packages and how to use them is hard, so it's primarily the technical coding that is difficult to pick up.

Bringing mathematical knowledge to the coding world and finding answers

coding :(

coding

Coding

I am confused about the definition of "no solution" for this course because it is not the mathematical definition

The coding. I have no background in coding besides CS1010E.

© Copyright EE, NUS. All Rights Reserved.

Arrangement of Python Clinic Sessions

- Python Clinic – Session 1 (Week 1 Fri, 16 Jan 7-8pm): Python basics
- Python Clinic – Session 2 (Week 2 Fri, 23 Jan 7-8pm): Data preprocessing, regression models & evaluation
- Python Clinic – Session 3 (Week 7 Fri, 6 March 8-9pm): Regression & Classification (Exam-Style Practice)
- Python Clinic – Session 4 (Week 8): Bias/Variance
- Python Clinic – Session 5 (Week 9): Gradient descent
- Python Clinic – Session 6 (Week 10): Decision tree and random forest
- Python Clinic – Session 7 (Week 11): Performance issues
- Python Clinic – Session 8 (Week 12): K-means clustering

Details (format and time) for Sessions 4–8 will be shared later.

© Copyright EE, NUS. All Rights Reserved.

Announcement

- **Additional Python Clinic Session 3 (Exam-Style Practice)**

- **Instructor:** Mr. Yunze Leng

- **Purpose:** Review essential Python libraries and work through typical questions from tutorials, the trial quiz, and past exam papers to help you prepare for the midterm.

- **When:** Friday, Week 7 (6 March) 8-9pm

* The recordings of the past two Python clinic sessions are available on Canvas.

© Copyright EE, NUS. All Rights Reserved.

Course Contents

- Introduction and Preliminaries (Yueming)
  - Introduction
  - Data Engineering
  - Introduction to Probability, Statistics, and Matrix

Module I

- Fundamental Machine Learning Algorithms I (Shaojing)
  - Systems of linear equations
  - Least squares, Linear regression
  - Polynomial regression, Ridge regression

Module II

- Fundamental Machine Learning Algorithms II (Shaojing)
  - Over-fitting, bias/variance trade-off
  - Optimization, Gradient descent
  - Decision Trees, Random Forest

Module III

- Performance and More Algorithms (Yueming)
  - Performance Issues
  - K-means Clustering
  - Neural Networks

© Copyright EE, NUS. All Rights Reserved.

Questions from Lecture 5

Qn 1: By adding a column of 1s to $\mathbf{X}$, are we manually specifying the bias term?

Ans: No, we are not manually setting the bias value.
When we add a column of 1s to $\mathbf{X}$, we are simply *rewriting the model* so that the bias becomes part of the weight vector $\mathbf{w}$, allowing the bias to be learned as part of the weight vector.

$$\mathbf{X}\mathbf{w} = \mathbf{y}$$

← Keep this format while incorporating a bias term.

$$\mathbf{X} = \begin{bmatrix} 1 & x_{1,1} & \dots & x_{1,d} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m,1} & \dots & x_{m,d} \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix},$$

$$\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \end{bmatrix}$$

← bias

```
# bias column manually included!
X = np.array([[1, -9], [1, -7], [1, -5], [1, 1], [1, 5], [1, 9]])
Y = np.array([[-6], [-6], [-4], [-1], [1], [4]])
w = inv(X.T @ X) @ X.T @ Y # Least squares solution
print(w)
```


© Copyright EE, NUS. All Rights Reserved.

Absorbing the offset/bias term b into the vector $\mathbf{w}$

$$\mathbf{X}\mathbf{w} = \mathbf{y}$$

$$f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$$

scalar b is called the offset (or bias)

$$f(x) = w_1x_1 + w_2x_2 + \cdots w_dx_d + b$$

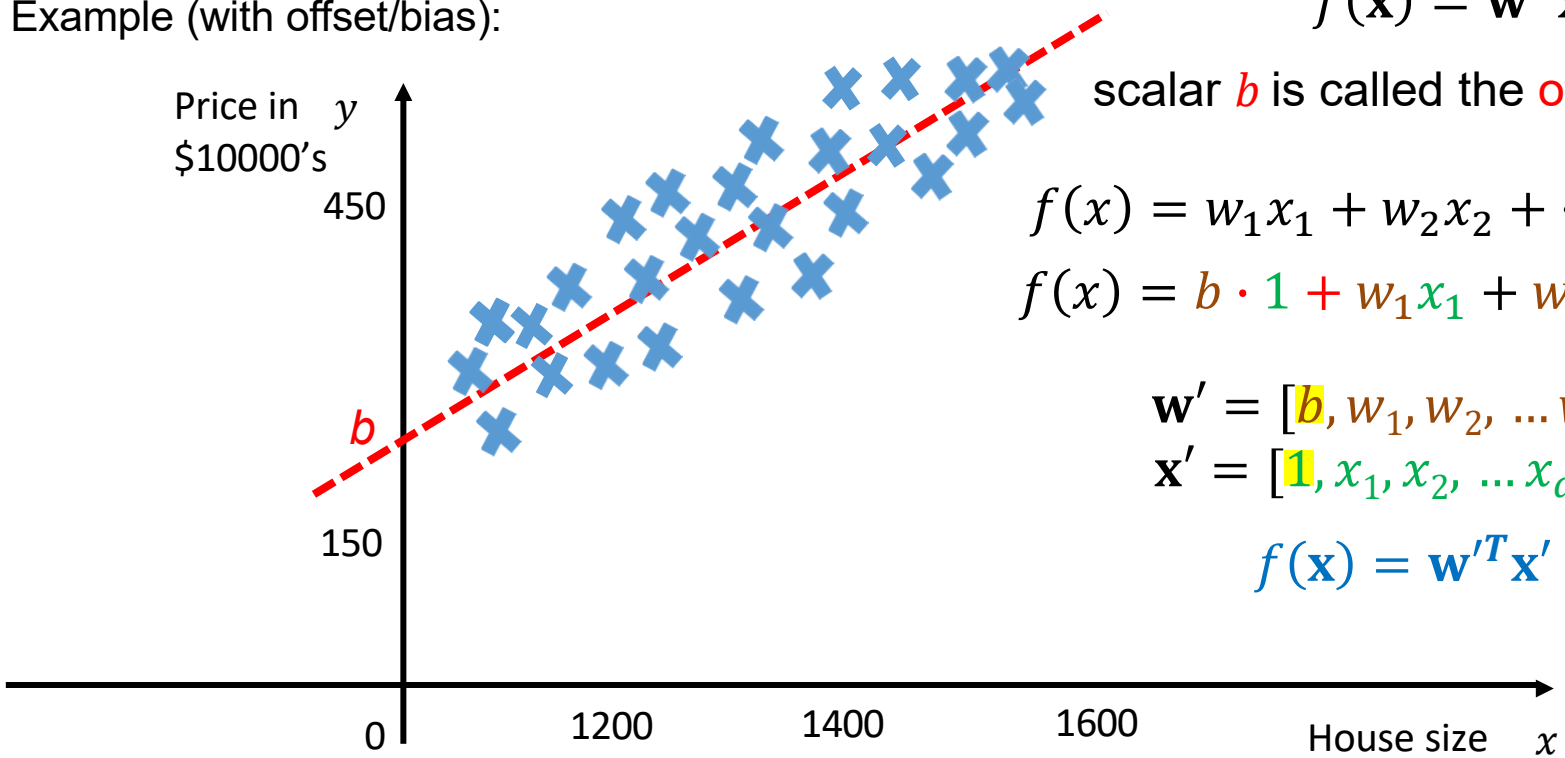
$$f(x) = b \cdot 1 + w_1x_1 + w_2x_2 + \cdots w_dx_d$$

$$\mathbf{w}' = [b, w_1, w_2, \ldots w_d]$$

$$\mathbf{x}' = [1, x_1, x_2, \ldots x_d]$$

$$f(\mathbf{x}) = \mathbf{w}'^T\mathbf{x}'$$

Example (with offset/bias):



© Copyright EE, NUS. All Rights Reserved.

Questions from Lecture 5

Qn 2: When a system is over- or under-determined, can the left or right inverse fail to exist, so we cannot apply the usual least-squares or least-norm formula?

Ans: Yes! This is one key reason we introduce **regularization** — which we will explain in this lecture.

The beautiful unification: Regularization For **any** system (over-, under-, or even-determined), the regularized solution is:

$$\hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\,\mathbf{X}^T\mathbf{y}$$

as long as $\lambda > 0$, this solution **always exists** and is **unique**, and serves as an **approximate** solution.

Qn 2': If we allow approximate solutions, does that mean there is never truly a "no solution" case?

Ans: Yes! In machine learning, we aim for the **best possible** solution, even if it's not exact. Small errors are expected— that's part of how ML works (we'll discuss how to evaluate the errors in Lec10).

	Even (m = d)	Over (m > d)	Under (m < d)
i) $\text{rank}(\mathbf{X}) = \text{rank}(\tilde{\mathbf{X}}) = d$	Unique sol: $\hat{\mathbf{w}} = \mathbf{X}^{-1}\mathbf{y}$	Unique sol: $\hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$	--
ii) $\text{rank}(\mathbf{X}) < \text{rank}(\tilde{\mathbf{X}})$	No sol	No sol in general, but when left inv $\mathbf{X}$ exists, least squares sol (approx.): $\hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$	No sol
iii) $\text{rank}(\mathbf{X}) = \text{rank}(\tilde{\mathbf{X}}) < d$	Infinitely many sol*	Infinitely many sol*	Infinitely many sol in general, but when right inv $\mathbf{X}$ exists, least norm sol: $\hat{\mathbf{w}} = \mathbf{X}^T(\mathbf{X}\mathbf{X}^T)^{-1}\mathbf{y}$

© Copyright EE, NUS. All Rights Reserved.

Correction: Typo in Lecture 5, slide #30

Example:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, \quad \frac{d\mathbf{Ax}}{d\mathbf{x}} = ?$$

2 x 1 vector function

$$\begin{cases} f_1(x_1, x_2, x_3) = a_{11}x_1 + a_{12}x_2 + a_{13}x_3 \\ f_2(x_1, x_2, x_3) = a_{21}\,x_1 + a_{22}x_2 + a_{23}x \end{cases}$$

2 x 3

$$\mathbf{f}(\mathbf{x}) = \mathbf{Ax} = \begin{bmatrix} f_1(x_1, x_2, x_3) \\ f_2(x_1, x_2, x_3) \end{bmatrix} = \begin{bmatrix} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 \end{bmatrix} \neq \begin{bmatrix} a_{11}x_1 & a_{12}x_2 & a_{13}x_3 \\ a_{21}x_1 & a_{22}x_2 & a_{23}x_3 \end{bmatrix}$$

$$\frac{d\mathbf{f}(\mathbf{x})}{d\mathbf{x}} = \frac{d\mathbf{Ax}}{d\mathbf{x}} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix} = \mathbf{A}$$

$\mathbf{f}(\mathbf{x})$ is a vector function of size $h \times 1$, $\mathbf{x}$ is a $d \times 1$ vector, then differentiation of $\mathbf{f}(\mathbf{x})$ results in a $h \times d$ matrix.

© Copyright EE, NUS. All Rights Reserved.

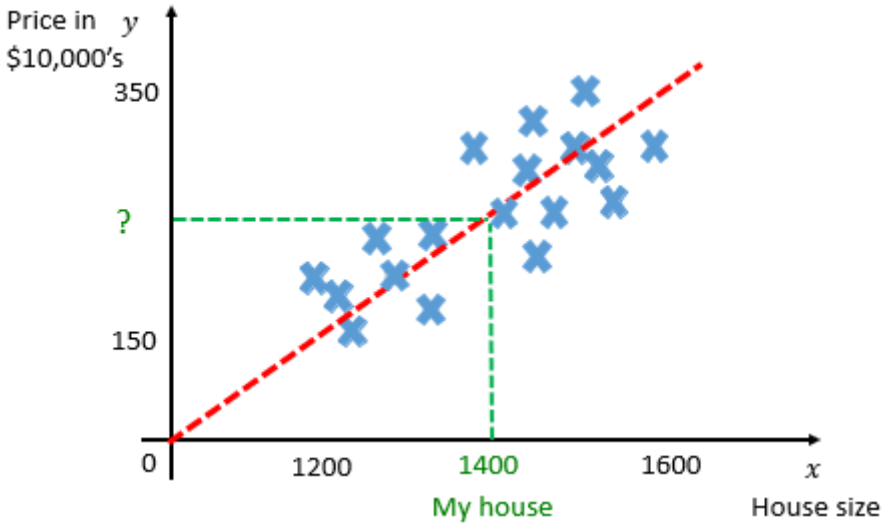
Agenda

- Linear Regression for Classification (Binary Classification, Multi-category Classification)
- Polynomial Regression
- Ridge Regression

© Copyright EE, NUS. All Rights Reserved.

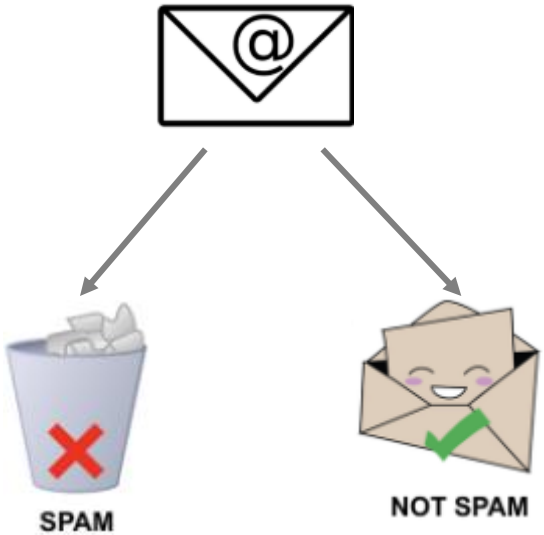
Recap: Regression vs Classification

Regression



y: continuous

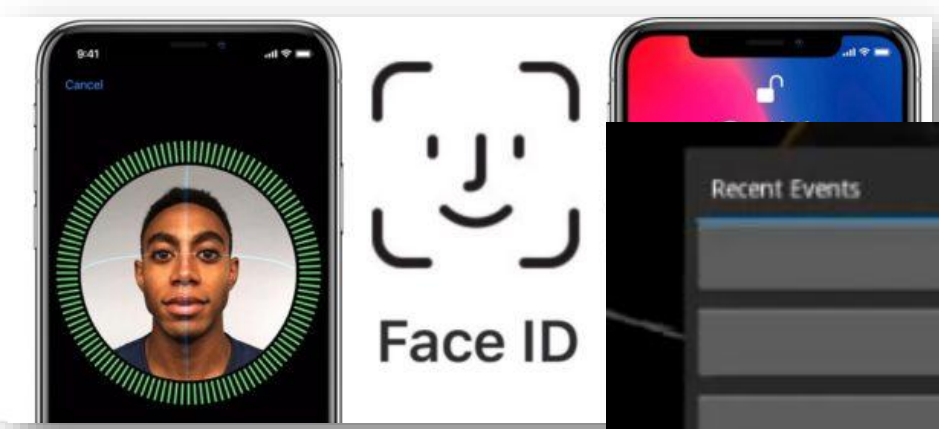
Classification

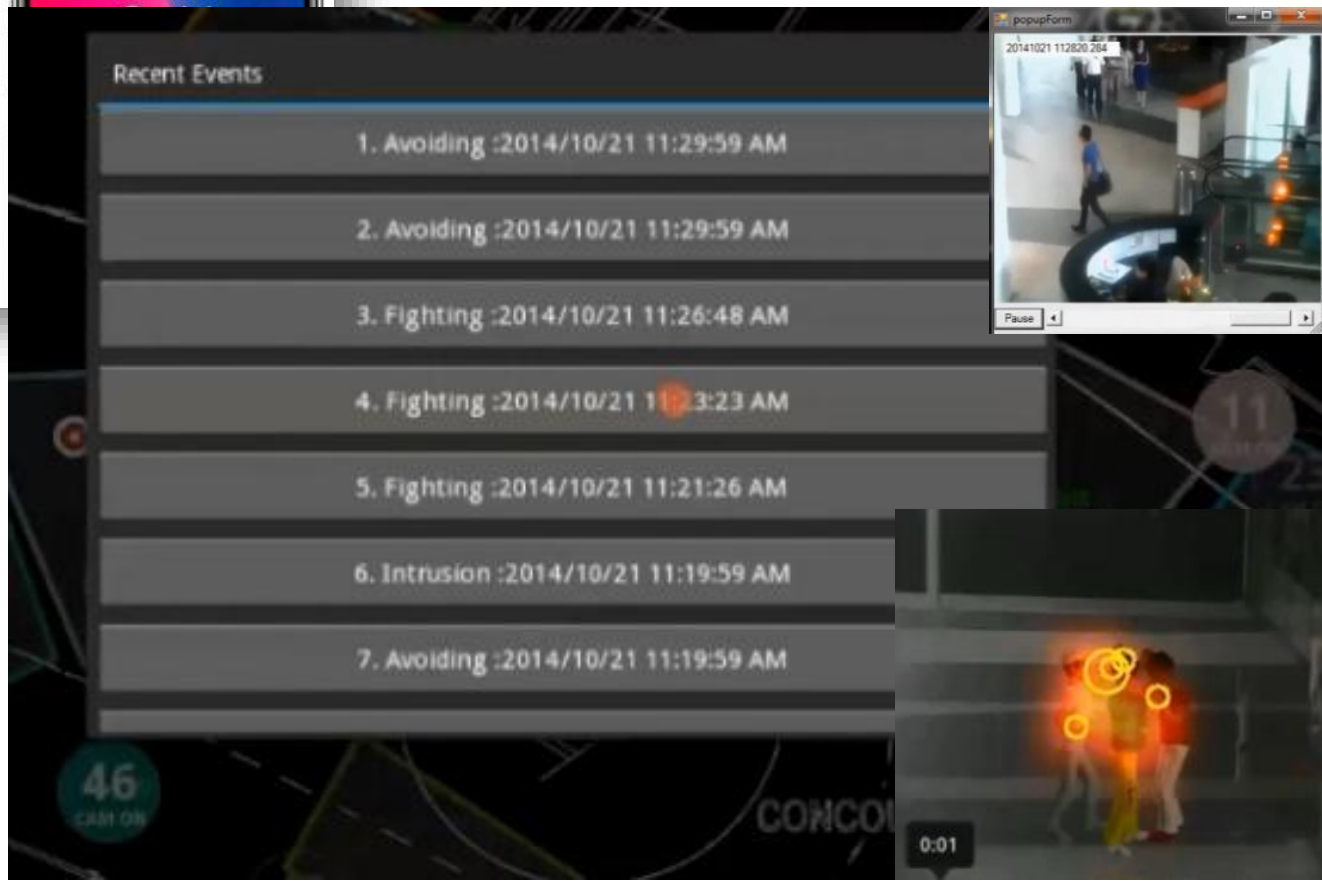


y: categorical/discrete

© Copyright EE, NUS. All Rights Reserved.

Linear Regression for Classification: Motivation

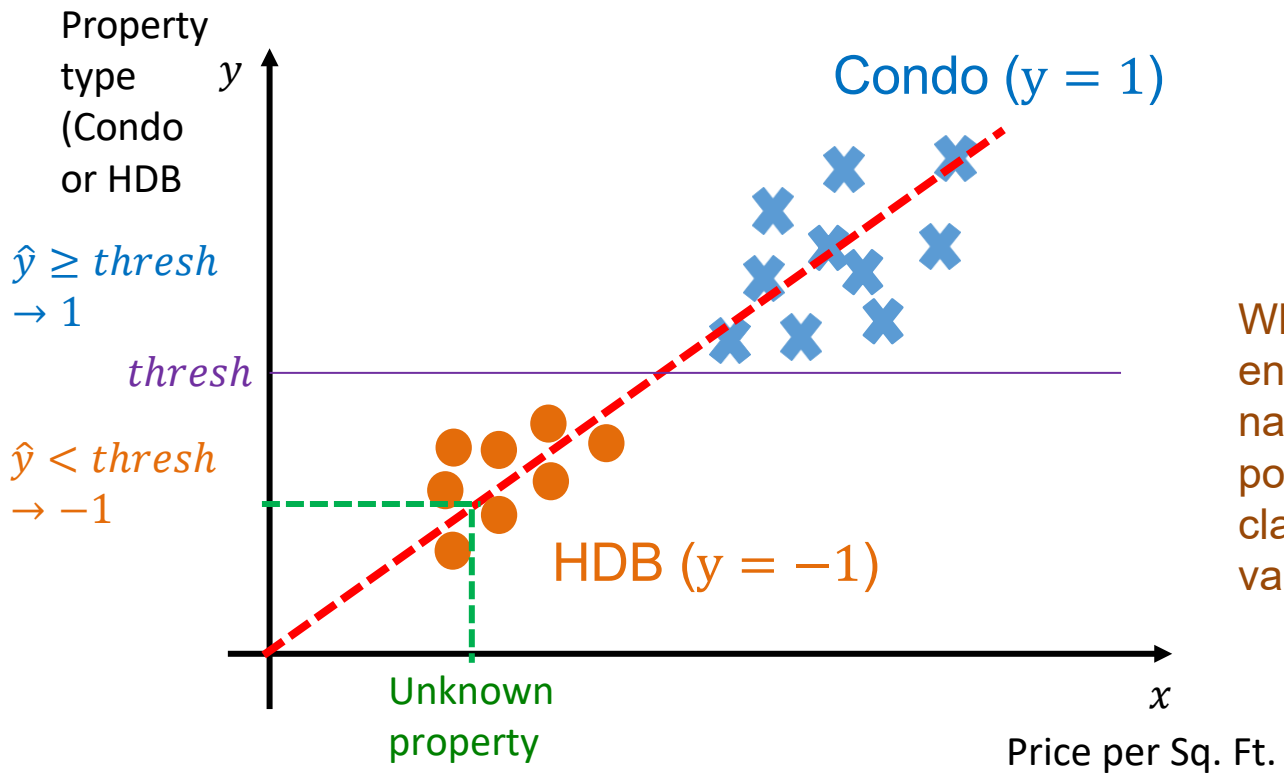




© Copyright EE, NUS. All Rights Reserved.

Linear Regression for Binary Classification: Overview

Toy example: Classify a property type into Condo or HDB based on price per sq. ft.

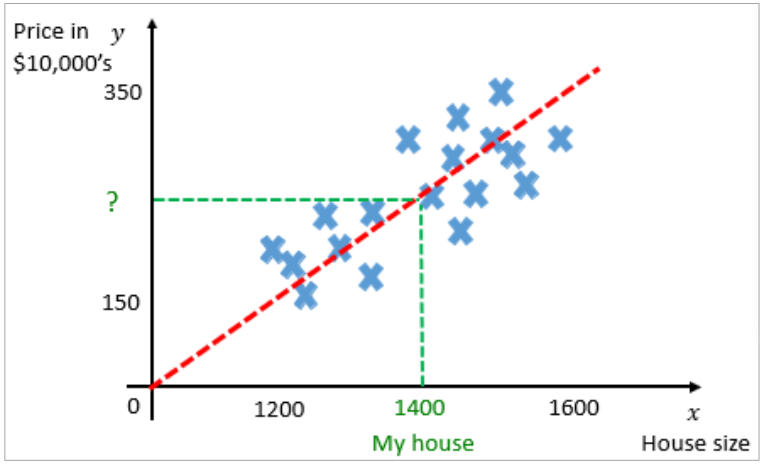


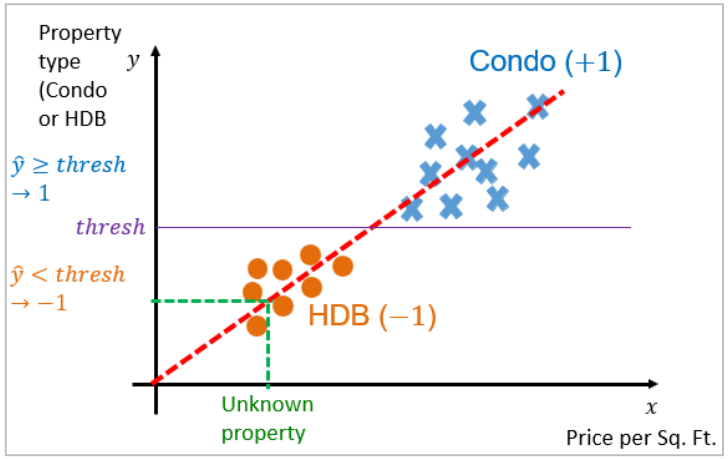
When binary classes are encoded as 1 and -1, the natural decision threshold is 0: positive values of $\hat{y}$ are classified as 1, and negative values as -1.

Ref: [Book4] Stephen Boyd and Lieven Vandenberghe, "Introduction to Applied Linear Algebra", Cambridge University Press, 2018 (chp.14)

© Copyright EE, NUS. All Rights Reserved.

Linear Regression for Binary Classification





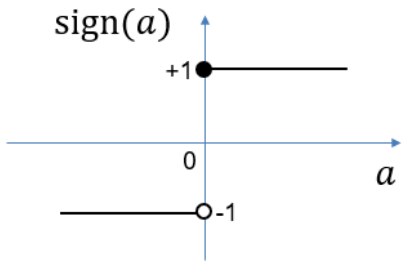
If using linear regression: $\hat{y} = \mathbf{w}^T\mathbf{x}$, however, given $\mathbf{x}$, we want $\hat{y} \in \{-1, 1\}$

continuous value $(-\infty, \infty)$ *discrete value*

Solution: Dichotomize $\hat{y}$ using the sign function (i.e., thresholding)

$$\hat{y} = sign(\mathbf{w}^T\mathbf{x})$$

$$sign(a) = \begin{cases} 1, & \text{if } a \geq 0 \\ -1, & \text{if } a < 0 \end{cases}$$



© Copyright EE, NUS. All Rights Reserved.

Linear Regression for Binary Classification: Learning & Prediction

$\mathbf{X}\mathbf{w} = \mathbf{y}$

If $\mathbf{X}^T\mathbf{X}$ is invertible, then

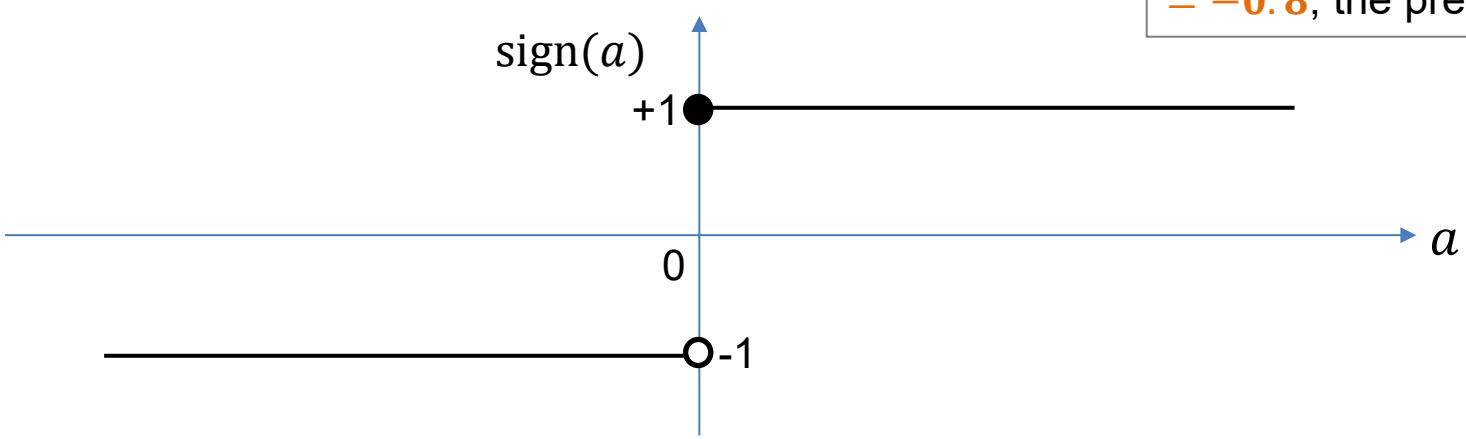
Learning: $\hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$, $\quad y_i \in \{-1, +1\}, i = 1, \ldots, m$

Prediction: $\hat{y}_{new} = \text{sign}(\mathbf{x}_{new}^T\hat{\mathbf{w}})$

$\mathbf{sign}(a) = +1$ for $a \geq 0$ and -1 for $a < 0$

E.g., If the raw prediction $\mathbf{x}_{new}^T\hat{\mathbf{w}} = \mathbf{0.2}$, the predicted class is $+\mathbf{1}$;

If the raw prediction $\mathbf{x}_{new}^T\hat{\mathbf{w}} = -\mathbf{0.8}$, the predicted class is $-\mathbf{1}$.



Ref: [Book4] Stephen Boyd and Lieven Vandenberghe, "Introduction to Applied Linear Algebra", Cambridge University Press, 2018 (chp.14)

© Copyright EE, NUS. All Rights Reserved.

Linear Regression for Binary Classification: Example (Training)

Training set $\{x_i, y_i\}_{i=1}^{m}$

$$\{x = -9\} \rightarrow \{y = -1\}$$
$$\{x = -7\} \rightarrow \{y = -1\}$$
$$\{x = -5\} \rightarrow \{y = -1\}$$
$$\{x = 1\} \rightarrow \{y = +1\}$$
$$\{x = 5\} \rightarrow \{y = +1\}$$
$$\{x = 9\} \rightarrow \{y = +1\}$$

$$\begin{array}{ccc} \mathbf{X} & \mathbf{w} & \mathbf{y} \end{array}$$

Bias column

$$\begin{bmatrix} 1 & -9 \\ 1 & -7 \\ 1 & -5 \\ 1 & 1 \\ 1 & 5 \\ 1 & 9 \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} = \begin{bmatrix} -1 \\ -1 \\ -1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

This set of linear equations has no exact solution.

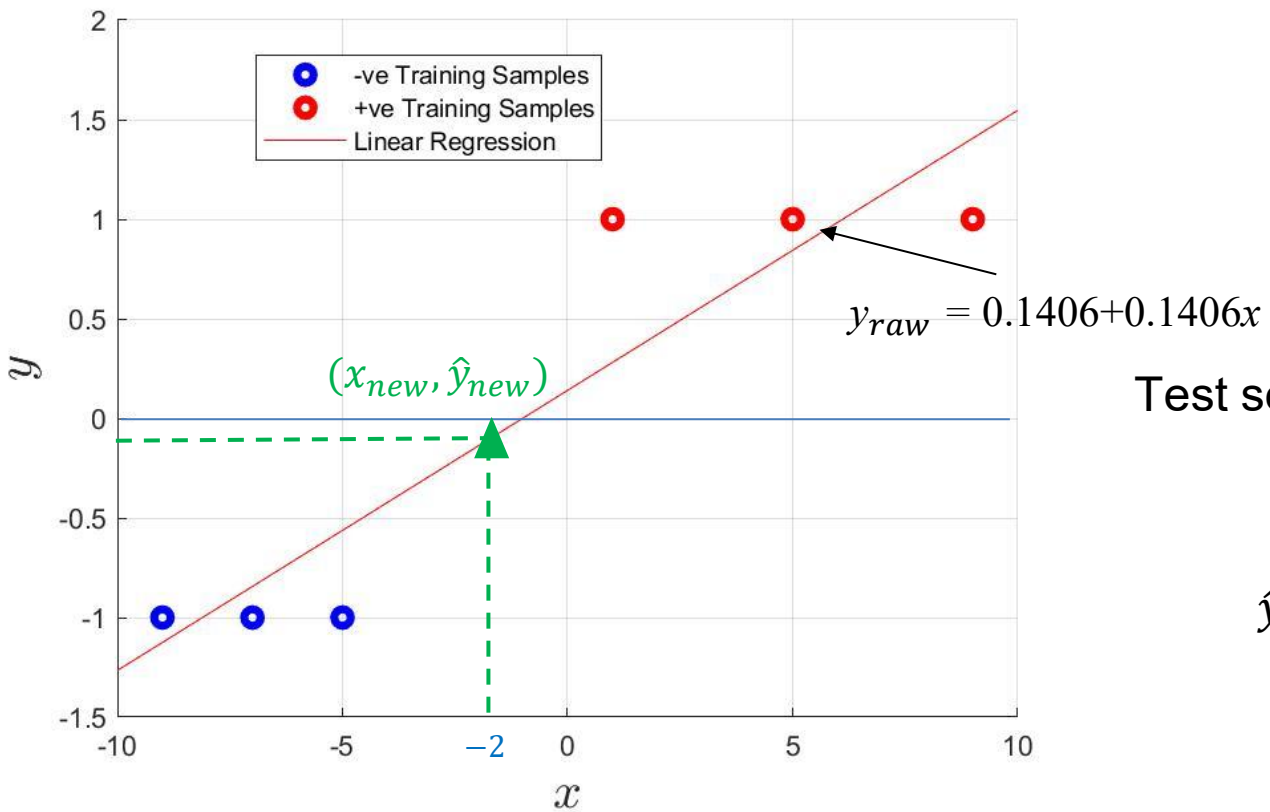
However, as $\mathbf{X}^T\mathbf{X}$ is invertible, we have the **least square approximation:**

$$\hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$
$$= \begin{bmatrix} 6 & -6 \\ -6 & 262 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ -9 & -7 & -5 & 1 & 5 & 9 \end{bmatrix} \begin{bmatrix} -1 \\ -1 \\ -1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.1406 \\ 0.1406 \end{bmatrix}$$

Similar to TUT6-Q4
Exam-style question

© Copyright EE, NUS. All Rights Reserved.

Linear Regression for Binary Classification: Example (Prediction)



Linear Regression for one-dimensional classification

Test set

$$\{x = -2\} \rightarrow \{y = ?\}$$

$$\hat{y}_{new} = \text{sign}(\mathbf{x}_{new}^T \hat{\mathbf{w}})$$

Bias column

$$= \text{sign}\left(\begin{bmatrix} 1 & -2 \end{bmatrix} \begin{bmatrix} 0.1406 \\ 0.1406 \end{bmatrix} \right)$$

$$= \text{sign}(-0.1406) = -1$$

© Copyright EE, NUS. All Rights Reserved.

Linear Regression for Binary Classification: Example (Python Demo)

```python
# EE2211 Lecture 6 Demo 1: Binary classification
import numpy as np
from numpy.linalg import inv

X = np.array([[1,-9], [1,-7], [1,-5], [1,1], [1,5], [1, 9]]) # bias column manually included
y = np.array([[-1], [-1], [-1], [1], [1], [1]])
## Linear regression for classification
w = inv(X.T @ X) @ X.T @ y
print("Estimated w")
print(w)

Xt = np.array([[1,-2]]) # bias column manually included
y_raw_predict = Xt @ w
print("Predicte y raw")
print(y_raw_predict)
y_class_predict = np.sign(y_raw_predict)
print("Predicted y class")
print(y_class_predict)
```

$$\hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

LSS

$$sign(a) = \begin{cases} 1, & \text{if } a \geq 0 \\ -1, & \text{if } a < 0 \end{cases}$$

- `np.sign()`: implement the sign function and dichotomize the predicted $\hat{y}$.

```
Estimated w
[[0.140625]
 [0.140625]]
Predicte y raw
[[-0.140625]]
Predicted y class
[[-1.]]
```

w_0

w_1

continuous

discrete

Ref: https://numpy.org/doc/2.2/reference/generated/numpy.sign.html

© Copyright EE, NUS. All Rights Reserved.

Linear Regression for Multi-Category Classification

$\mathbf{X}\mathbf{w} = \mathbf{Y}$

If $\mathbf{X}^T\mathbf{X}$ is invertible, then

one-hot encoded Y

Learning: $\widehat{\mathbf{W}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}$, $\mathbf{Y} \in \mathbb{R}^{m\times C}$

Prediction: $\hat{y}_{new} = \arg\max_{k=1,\dots,C}\left(\mathbf{x}_{new}^T\widehat{\mathbf{W}}(:,k)\right)$ for each $\mathbf{x}_{new}^T$ of $\mathbf{X}_{new}$

Each row (of $i = 1, \dots, m$) in $\mathbf{Y}$ has a **one-hot** encoding/assignment:

e.g., If sample i has class 1, its label vector is $\mathbf{y}_i = [1, 0, 0, \dots, 0]$

If sample i has class 2, its label vector is $\mathbf{y}_i = [0, 1, 0, \dots, 0]$

If sample i has class C, its label vector is $\mathbf{y}_i = [\underbrace{0, 0, \dots, 0, 1}_{\text{C (total no. of classes)}}]$

In $\mathbf{Y} \in \mathbb{R}^{m\times C}$, the column position of the **"1"** indicates the class each sample belongs to.

Ref: Hastie, Tibshirani, Friedman, "The Elements of Statistical Learning", (2nd ed., 12th printing) 2017 (chp.4)

© Copyright EE, NUS. All Rights Reserved.

Recap: One-hot Encoding

Apple: 0
Banana: 1
Watermelon: 2

Fruit type	Apple	Banana	Watermelon	One-hot encoding
Apple	1	0	0	[1, 0, 0]
Banana	0	1	0	[0, 1, 0]
Watermelon	0	0	1	[0, 0, 1]

In one-hot encoding, each category in a dataset is represented as a *binary* vector, with the vector length equal to the **number of unique categories**. Each vector contains ***all zeros except*** for a **single "1"** at the position corresponding to that category.

Advantages of one-hot encoding over numerical values:

- **Prevents false ordering:**

Using numeric labels (e.g., 0 = apple, 1 = banana, 2 = watermelon) can wrongly imply an order or hierarchy. One-hot encoding removes this by representing each category as an independent binary feature.

- **Ensures equal distance between categories:**

Many algorithms rely on distances between values. One-hot encoding ensures all categories are equally separated, avoiding biased similarity assumptions.

© Copyright EE, NUS. All Rights Reserved.

Linear Regression for Multi-Category Classification

Fruit type	Apple	Banana	Watermelon	One-hot encoding
Apple	1	0	0	[1, 0, 0]
Banana	0	1	0	[0, 1, 0]
Watermelon	0	0	1	[0, 0, 1]

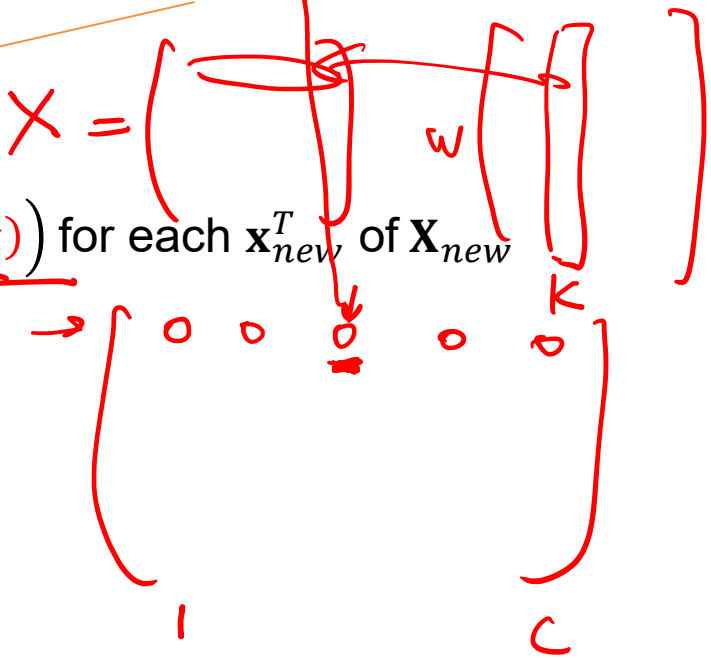
one-hot encoded Y

$\mathbf{X}\mathbf{w} = \mathbf{Y}$ **Prediction**: $\widehat{\mathbf{Y}}_{raw} = \mathbf{X}_{new}\widehat{\mathbf{W}}$

If $\mathbf{X}^T\mathbf{X}$ is invertible, then

Learning: $\widehat{\mathbf{W}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}$, $\mathbf{Y} \in \mathbf{R}^{m\times c}$

Prediction: $\hat{y}_{new} = \arg\max_{k=1,\ldots,c} \left(\mathbf{x}_{new}^T \widehat{\mathbf{W}}(:,k)\right)$ for each $\mathbf{x}_{new}^T$ of $\mathbf{X}_{new}$



© Copyright EE, NUS. All Rights Reserved.

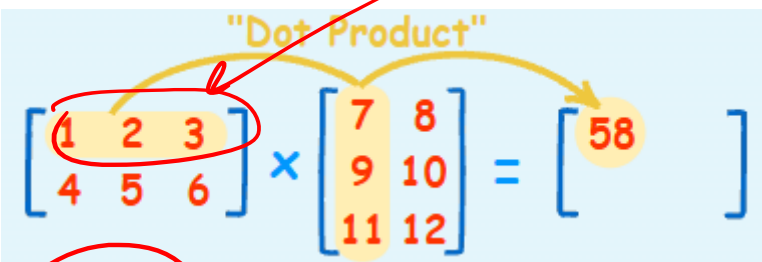
Linear Regression for Multi-Category Classification

If $\mathbf{X}^T\mathbf{X}$ is invertible, then

Learning: $\widehat{\mathbf{W}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}, \quad \mathbf{Y} \in \mathbf{R}^{m\times C}$

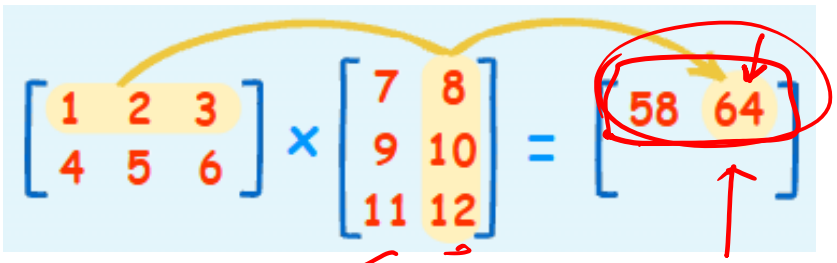
Prediction: $\hat{y}_{new} = \arg\max_{k=1,\ldots,C}\left(\mathbf{x}_{new}^T\underline{\widehat{\mathbf{W}}(:,k)}\right)$ for each $\mathbf{x}_{new}^T$ of $\mathbf{X}_{new}$

$\mathbf{X}$ $\widehat{\mathbf{W}}$ $\widehat{\mathbf{Y}}$



$k = 1$

$\mathbf{X}$ $\widehat{\mathbf{W}}$ $\widehat{\mathbf{Y}}$



$k = 2$

#2
class 2

© Copyright EE, NUS. All Rights Reserved.

Linear Regression for Multi-Category Classification

Fruit type	Apple	Banana	Watermelon	One-hot encoding
Apple	1	0	0	[1, 0, 0]
Banana	0	1	0	[0, 1, 0]
Watermelon	0	0	1	[0, 0, 1]

one-hot encoded Y

$\mathbf{X}\mathbf{w} = \mathbf{Y}$ **Prediction**: $\widehat{\mathbf{Y}}_{raw} = \mathbf{X}_{new}\widehat{\mathbf{W}}$

If $\mathbf{X}^T\mathbf{X}$ is invertible, then

Learning: $\widehat{\mathbf{W}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}$, $\mathbf{Y} \in \mathbf{R}^{m\times C}$

Prediction: $\hat{y}_{new} = \arg\max_{k=1,\dots,C}\left(\mathbf{x}_{new}^T\widehat{\mathbf{W}}(:,k)\right)$ for each $\mathbf{x}_{new}^T$ of $\mathbf{X}_{new}$

E.g.

$$\widehat{\mathbf{Y}}_{raw} = \mathbf{X}_{new}\widehat{\mathbf{W}}$$

$$\begin{array}{l} \text{sample 1} \rightarrow \\ \text{sample 2} \rightarrow \\ \text{sample 3} \rightarrow \end{array} = \begin{array}{c} \begin{bmatrix} 9 & 12 & 0.7 \\ 80 & -1 & 7.5 \\ -0.6 & -2 & 0.2 \end{bmatrix} \\ \begin{array}{ccc} \uparrow & \uparrow & \uparrow \end{array} \end{array} \Rightarrow \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix} \begin{array}{l} \rightarrow \text{Class 2} \\ \rightarrow \text{Class 1} \\ \rightarrow \text{Class 3} \end{array}$$

Class 1 Class 2 Class 3

(based on the one-hot encoding)

During prediction, The values $\widehat{\mathbf{Y}}_{raw}$ are **scores** for each class, higher means the model thinks that class is more likely.

For each row, we select the **class with the highest score** as the predicted label.

© Copyright EE, NUS. All Rights Reserved.

Linear Regression for Multi-Category Classification: Example (preprocessing)

Task: Classify an animal as **cat**, **dog**, or **bird**.

Our $m = 4$ feature vectors are

$$x_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad x_2 = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \quad x_3 = \begin{bmatrix} 1 \\ 3 \end{bmatrix} \quad x_4 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

The raw classes (there are $C = 3$ of them) are: $y_1 = cat,\quad y_2 = dog,\quad y_3 = cat,\quad y_4 = bird.$

One-hot encoding:

Animal type	One-hot encoding
Cat	[1, 0, 0]
Dog	[0, 1, 0]
Bird	[0, 0, 1]

cat dog bird

$$\mathbf{y}_1 = [1 \;\; 0 \;\; 0], \quad \mathbf{y}_2 = [0 \;\; 1 \;\; 0],$$
$$\mathbf{y}_3 = [1 \;\; 0 \;\; 0], \quad \mathbf{y}_4 = [0 \;\; 0 \;\; 1].$$

$$\mathbf{Y} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{array}{l} \leftarrow \text{sample 1} \\ \leftarrow \text{sample 2} \\ \leftarrow \text{sample 3} \\ \leftarrow \text{sample 4} \end{array}$$

Class 1 Class 2 Class 3
(based on the one-hot encoding)

Similar to TUT6-Q5
Exam-style question

Python demo

© Copyright EE, NUS. All Rights Reserved.

Linear Regression for Multi-Category Classification: Example (Training)

$$\begin{array}{cccc} & \mathbf{X} & \mathbf{W} & \mathbf{Y} \text{ (one-hot encoded)} \\ \text{Bias column} \rightarrow & \begin{bmatrix} 1 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & 1 & 3 \\ 1 & 1 & 0 \end{bmatrix} & \begin{bmatrix} w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,1} & w_{2,2} & w_{2,3} \\ w_{3,1} & w_{3,2} & w_{3,3} \end{bmatrix} = & \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \end{array}$$

This set of linear equations has no exact solution.

However, as $\mathbf{X}^T\mathbf{X}$ is invertible, we have the **least square approximation:**

$$\hat{\mathbf{W}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}$$

$$= \begin{bmatrix} 4 & 2 & 5 \\ 2 & 4 & 3 \\ 5 & 3 & 11 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 1 \\ 1 & 1 & 3 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0.5 & 0.5 \\ 0.2857 & -0.5 & 0.2143 \\ 0.2857 & 0 & -0.2857 \end{bmatrix}$$

© Copyright EE, NUS. All Rights Reserved.

Linear Regression for Multi-Category Classification: Example (Prediction)

Test set $\mathbf{X}_{new}$ $\{x_1 = 6, \quad x_2 = 8\} \rightarrow \{class\ 1, 2, or\ 3?\}$

$\{x_1 = 0, \ x_2 = -1\} \rightarrow \{class\ 1, 2, or\ 3?\}$

$$\hat{\mathbf{Y}}_{raw} = \mathbf{X}_{new}\hat{\mathbf{W}} = \begin{bmatrix} 1 & 6 & 8 \\ 1 & 0 & -1 \end{bmatrix} \begin{bmatrix} 0 & 0.5 & 0.5 \\ 0.2857 & -0.5 & 0.2143 \\ 0.2857 & 0 & -0.2857 \end{bmatrix}$$

$$= \begin{bmatrix} 4 & -2.50 & -0.50 \\ -0.2587 & 0.50 & 0.7857 \end{bmatrix}$$

$\hat{y}_{new} = \arg\max_{k=1,\ldots,C} \left(\mathbf{x}_{new}^T \hat{\mathbf{W}}(:,k)\right)$ for each $\mathbf{x}_{new}^T$ of $\mathbf{X}_{new}$

$$\Rightarrow \arg\max_{k=1,\ldots,C}\left(\begin{bmatrix} 4 & -2.50 & -0.50 \\ -0.2587 & 0.50 & 0.7857 \end{bmatrix}\right)$$

$$\Rightarrow \begin{bmatrix} 1 \\ 3 \end{bmatrix}$$

Class 1

Class 3

For each row of $\hat{\mathbf{Y}}_{raw}$, the **column position** of the **largest** number (across all columns for that row) determines the **class label**.
E.g. in the 1st row, the maximum number is 4 which is in column 1. So the resulting predicted class is 1.

© Copyright EE, NUS. All Rights Reserved.

Linear Regression for Multi-Category Classification: Example (Python Demo)

```python
# EE2211 Lecture 6 Demo 2 Multi-class classification
import numpy as np
from numpy.linalg import inv
from sklearn.preprocessing import OneHotEncoder
X = np.array([[1, 1, 1], [1, -1, 1], [1, 1, 3], [1, 1, 0]])
y_class = np.array([[1], [2], [1], [3]])
y_onehot = np.array([[1, 0, 0], [0, 1, 0], [1, 0, 0], [0, 0, 1]])
print("One-hot encoding manual")
print(y_class)
print(y_onehot)

print("One-hot encoding function")
onehot_encoder=OneHotEncoder(sparse_output=False)
print(onehot_encoder)
Ytr_onehot = onehot_encoder.fit_transform(y_class)
print(Ytr_onehot)

## Linear Classification
print("Estimated W")
W = inv(X.T @ X) @ X.T @ Ytr_onehot
print(W)

## Prediction
X_test = np.array([[1, 6, 8], [1, 0, -1]])
yt_est = X_test@W;
print("Test")
print(yt_est)
yt_class = [[1 if y == max(x) else 0 for y in x] for x in yt_est ]
print("class label test")
print(yt_class)
```

Used to convert categorical target variables into one-hot encoding format.

bias column manually included

Convert y into one-hot encoded vectors

```
Ytr_onehot
[[1. 0. 0.]
 [0. 1. 0.]
 [1. 0. 0.]
 [0. 0. 1.]]
```

$\hat{y}_{new} = \arg\max_{k=1,\ldots,C} \left(\mathbf{x}_{new}^T \widehat{\mathbf{W}}(:,k)\right)$ for each $\mathbf{x}_{new}^T$ of $\mathbf{X}_{new}$

Quick Question 1

True or False: Suppose we want to build a single linear regression model where each sample has **3 input features**, and the goal is to predict **one of 3 classes** for each sample. The model has **9 parameters to learn**.

$d = 3$

$C = 3$

PollEv.com/shaojing

$f_1 \quad f_2 \quad f_3$

$C_1 \quad C_2 \quad C_3$

$w_{11} f_1 + w_{12} f_2 + w_{13} f_3 + b_1 = C_1$

$w_{21} f_1 + w_{22} f_2 + w_{23} f_3 + b_2 = C_2$

$\vdots$

Exam-style question

© Copyright EE, NUS. All Rights Reserved.

Quick Question 1

True or False: Suppose we want to build a single linear regression model with **three input features** to predict **three different classes**. The model has **9 parameters to learn**.

Ans:

- Each class has its own set of weights: one weight per feature.
- With 3 features and 3 classes:

$$3 \textit{ classes} \times 3 \textit{ features} = 9 \textit{ feature weights}$$

- Bias terms are also learned separately for each class.
  - 1 bias per class × 3 classes = 3 biases
- Total parameters (weights + biases) = 9 + 3 = 12

Reminder: Don't forget the bias when counting parameters!

The number of parameters (dimensions of $\mathbf{W}$ or $\mathbf{w}$) depends on the number of features (and prediction classes), not on the no. of training samples.

Example with 5 samples:

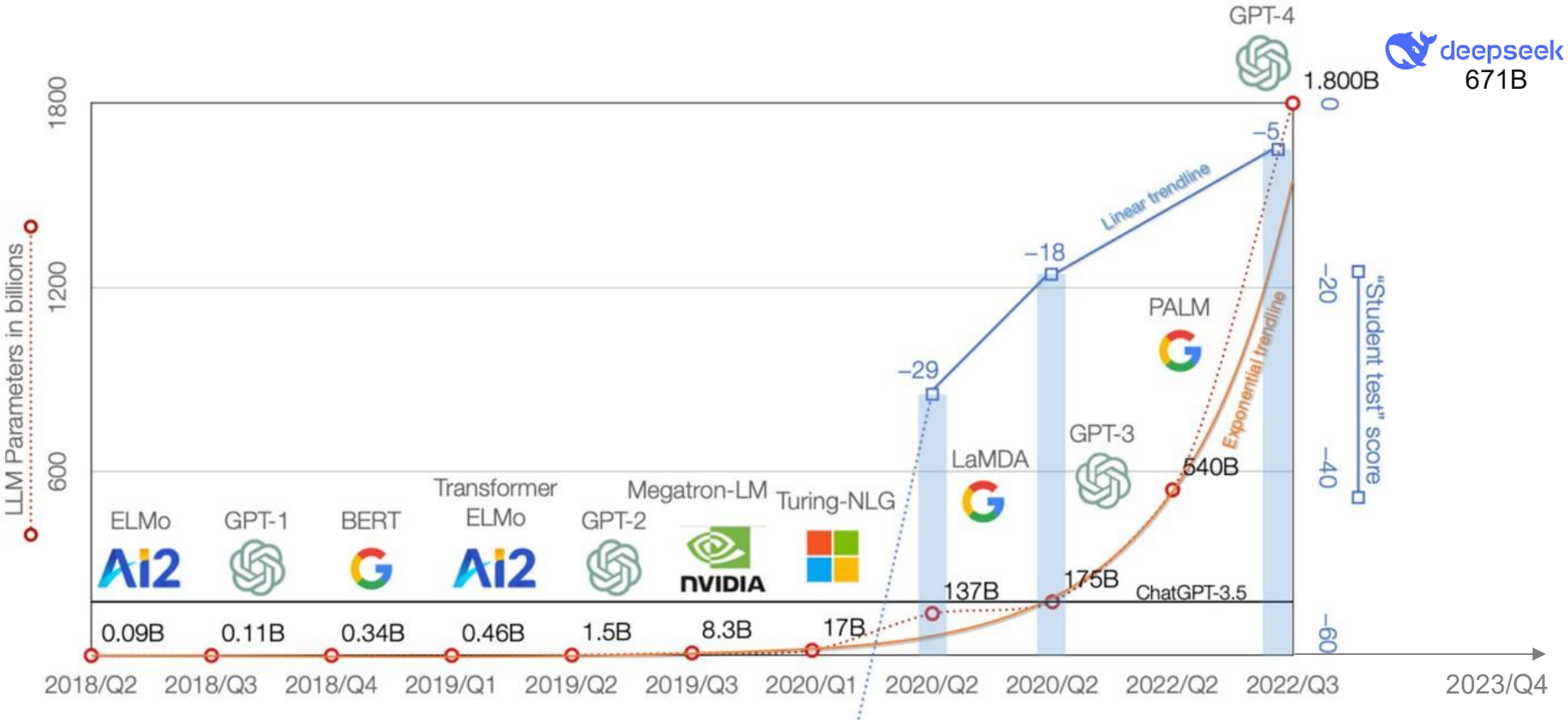
$$\begin{array}{ccc} & \mathbf{X} & \mathbf{W} & \mathbf{Y} \\ \text{Bias column} \rightarrow & \begin{bmatrix} 1 & 1 & 1 & 2 \\ 1 & -1 & 1 & -2 \\ 1 & 1 & -2 & -1 \\ 1 & 1 & 0 & 1 \\ 1 & -1 & 1 & 2 \end{bmatrix} & \begin{bmatrix} w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,1} & w_{2,2} & w_{2,3} \\ w_{3,1} & w_{3,2} & w_{3,3} \\ w_{4,1} & w_{4,2} & w_{4,3} \end{bmatrix} & = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} \end{array}$$

© Copyright EE, NUS. All Rights Reserved.

Bonus Concept: Why the Number of Model Parameters Matters

- **Indicates model size**: The number of parameters shows how big the model is. More parameters mean a larger and more flexible model

- **Reflects model power**: More parameters allow the model to capture more



Source: https://www.mdpi.com/1999-5903/15/10/336

© Copyright EE, NUS. All Rights Reserved.

© Copyright EE, NUS. All Rights Reserved.

Agenda

- Linear Regression for Classification
- Polynomial Regression (a more powerful extension of linear regression)
- Ridge Regression

© Copyright EE, NUS. All Rights Reserved.

Polynomial Regression: Motivation

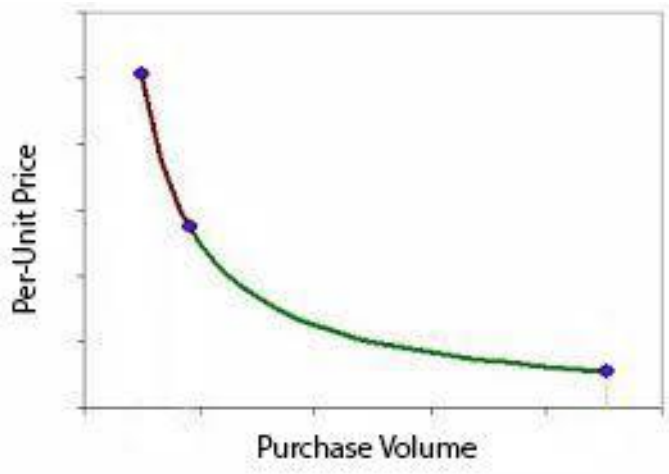
Price doesn't increase in a strictly linear way with size.

	PERSONAL	SMALL	MEDIUM	LARGE	STUFFED CRUST
SUPREME	$6.80	$17.80	$22.20	$26.30	$29.30
SUPER SUPREME	$6.80	$18.70	$25.10	$27.60	$30.60
CHEESE LOVER'S	$6.80	$11.70	$16.70	$20.50	$23.50
VEGGIE LOVER'S	$6.80	$17.20	$22.20	$26.30	$29.30
MEAT LOVER'S	$6.80	$18.30	$25.10	$27.60	$30.60
PEPPERONI LOVER'S	$6.80	$15.50	$20.50	$24.30	$27.30
HAWAIIAN	$6.80	$14.70	$18.30	$21.90	$24.90
BBQ CHICKEN	$6.80	$14.70	$18.30	$21.90	$24.90
CHICKEN SUPREME	$6.80	$17.20	$22.20	$26.30	$29.30

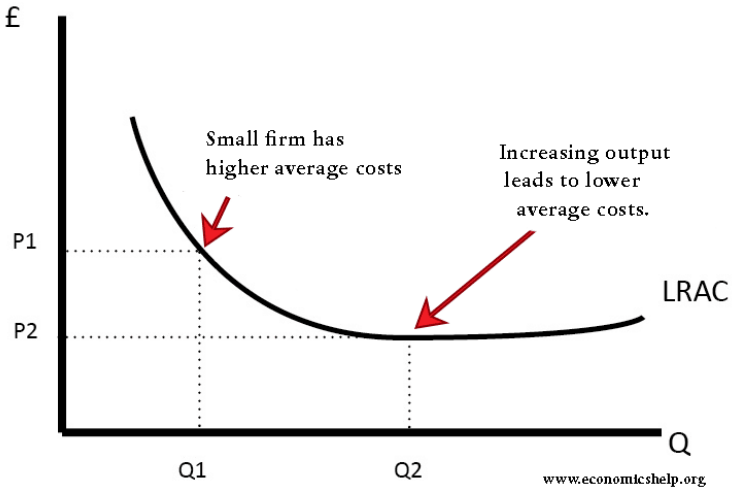
© Copyright EE, NUS. All Rights Reserved.

Polynomial Regression: Motivation

The unit sell price is not linear



Economies of scale



Reference: https://www.economicshelp.org/microessays/costs/economies-scale/

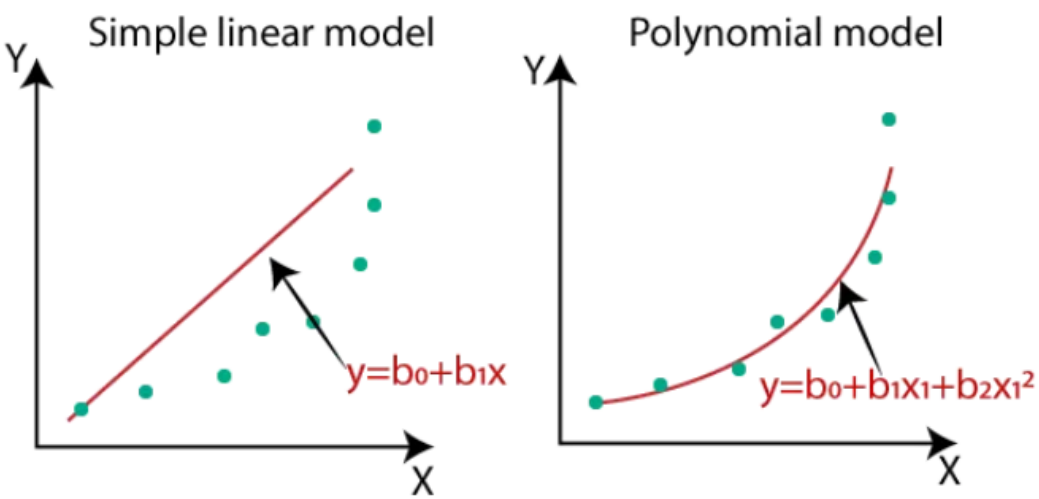
© Copyright EE, NUS. All Rights Reserved.

Polynomial Regression: Motivation

Polynomial regression models non-linear relationships by fitting a curve using a polynomial function.

- E.g. a polynomial function of degree = 2 with two input features (x_1, x_2) is:

$$f_{\mathbf{w}}(\mathbf{x}) = w_0 + w_1 x_1 + w_2 x_2 + w_{12}\, x_1 x_2 + w_{11}\, x_1^2 + w_{22}\, x_2^2.$$



Why Polynomial: *polynomial regression improves modeling power by* ***creating new features**** *that capture nonlinear relationships.*

*Essentially, we create new features from the original inputs

© Copyright EE, NUS. All Rights Reserved.

Polynomial Regression: Model

Generalized Linear Discriminant Function:

$$f_{\mathbf{w}}(\mathbf{x}) = w_0 + \sum_{i=1}^{d} w_i\, x_i + \sum_{i=1}^{d}\sum_{j=1}^{d} w_{ij}\, x_i x_j + \sum_{i=1}^{d}\sum_{j=1}^{d}\sum_{k=1}^{d} w_{ijk}\, x_i x_j x_k + \cdots$$

Switch to matrix notation: $\boldsymbol{f}_{\mathbf{w}}(\mathbf{x}) = \mathbf{P}\mathbf{w} = \begin{bmatrix} \boldsymbol{p}_1^T \mathbf{w} \\ \vdots \\ \boldsymbol{p}_m^T \mathbf{w} \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \\ \vdots \\ w_{ij} \\ \vdots \\ w_{ijk} \\ \vdots \end{bmatrix}$

where $\boldsymbol{p}_l^T \mathbf{w} = [1, x_{l,1}, \ldots, x_{l,d}, \ldots, x_{l,i}x_{l,j}, \ldots, x_{l,i}x_{l,j}x_{l,k}, \ldots]$

$l = 1, \ldots, m$; *d* denotes the dimension of input features;
m denotes the number of samples

Ref: Duda, Hart, and Stork, "Pattern Classification", 2001 (Chp.5)

© Copyright EE, NUS. All Rights Reserved.

Polynomial Regression: Model (cont.)

Linear regression: $\mathbf{X}\mathbf{w} = \mathbf{y}$,

where $\mathbf{X}$ contains the original input features, plus a bias column

Polynomial regression: $\mathbf{P}\mathbf{w} = \mathbf{y}$,

where $\mathbf{P}$ is the feature matrix with polynomial features of the original inputs, including the bias column.

For example, suppose we have **4 samples**, each with a **single feature**, and we want to perform **2nd-order polynomial regression**.

$m = 4$ $d = 1$

$$\mathbf{P} = \begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \\ 1 & x_4 & x_4^2 \end{bmatrix}$$

© Copyright EE, NUS. All Rights Reserved.

Polynomial Regression: Training & Prediction

In polynomial regression, we replace the original features with a polynomial feature matrix $\mathbf{P}$, giving $\mathbf{P}\mathbf{w} = \mathbf{y}$, and then solve it using the same methods as linear regression.

$X \to P$

For Regression Applications

- Learning (continuous y): $\hat{\mathbf{w}} = (\mathbf{P}^T\mathbf{P})^{-1}\mathbf{P}^T\mathbf{y}$
- Prediction: $\hat{f}_{\mathbf{w}}(\mathbf{P}_{new}) = \mathbf{P}_{new}\hat{\mathbf{w}}$

For Classification Applications

- Learning (discrete y ($y \in \{-1, +1\}$)): $\hat{\mathbf{w}} = (\mathbf{P}^T\mathbf{P})^{-1}\mathbf{P}^T\mathbf{y}$
- Binary Prediction: $\hat{f}^C_{\mathbf{w}}(\mathbf{P}_{new}) = \mathrm{sign}(\mathbf{P}_{new}\hat{\mathbf{w}})$
- Multi-Category Prediction: $\hat{f}^C_{\mathbf{w}}(\mathbf{P}_{new}) = \arg\max_{k=1,\ldots,C}(\mathbf{P}_{new}\hat{\mathbf{W}}(:,k))$

© Copyright EE, NUS. All Rights Reserved.

Polynomial Regression: Example

Training set $\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^{m}$

$\{x_1 = 0, x_2 = 0\} \rightarrow \{y = -1\}$ ← sample 1

$\{x_1 = 1, x_2 = 1\} \rightarrow \{y = -1\}$ ← sample 2

$\{x_1 = 1, x_2 = 0\} \rightarrow \{y = +1\}$ ← sample 3

$\{x_1 = 0, x_2 = 1\} \rightarrow \{y = +1\}$ ← sample 4

Perform **2nd order** polynomial regression for *binary classification*.

$$f_{\mathbf{w}}(\mathbf{x}) = w_0 + w_1 x_1 + w_2 x_2 + w_{12}\, x_1 x_2 + w_{11}\, x_1^2 + w_{22}\, x_2^2$$

$$= \begin{bmatrix} 1 & x_1 & x_2 & x_1 x_2 & x_1^2 & x_2^2 \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ w_{12} \\ w_{11} \\ w_{22} \end{bmatrix}$$

Stack the 4 training samples, create new features using the original inputs:

$$\mathbf{P} = \begin{bmatrix} 1 & x_{1,1} & x_{1,2} & x_{1,1}x_{1,2} & x_{1,1}^2 & x_{1,2}^2 \\ 1 & x_{2,1} & x_{2,2} & x_{2,1}x_{2,2} & x_{2,1}^2 & x_{2,2}^2 \\ 1 & x_{3,1} & x_{3,2} & x_{3,1}x_{3,2} & x_{3,1}^2 & x_{3,2}^2 \\ 1 & x_{4,1} & x_{4,2} & x_{4,1}x_{4,2} & x_{4,1}^2 & x_{4,2}^2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Similar to TUT6-Q4
Exam-style question

© Copyright EE, NUS. All Rights Reserved.

Polynomial Regression: Example (training)

2nd order polynomial model

$$\mathbf{P} = \begin{bmatrix} 1 & x_{1,1} & x_{1,2} & x_{1,1}x_{1,2} & x_{1,1}^2 & x_{1,2}^2 \\ 1 & x_{2,1} & x_{2,2} & x_{2,1}x_{2,2} & x_{2,1}^2 & x_{2,2}^2 \\ 1 & x_{3,1} & x_{3,2} & x_{3,1}x_{3,2} & x_{3,1}^2 & x_{3,2}^2 \\ 1 & x_{4,1} & x_{4,2} & x_{4,1}x_{4,2} & x_{4,1}^2 & x_{4,2}^2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

P X

This set of linear equations (under-determined system) has infinite solutions.

As $\mathbf{P}\mathbf{P}^T$ is invertible, we have the **least norm solution:**

$$\hat{\mathbf{w}} = \mathbf{P}^T(\mathbf{P}\mathbf{P}^T)^{-1}\mathbf{Y} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 6 & 3 & 3 \\ 1 & 3 & 3 & 1 \\ 1 & 3 & 1 & 3 \end{bmatrix}^{-1} \begin{bmatrix} -1 \\ -1 \\ +1 \\ +1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \\ 1 \\ -4 \\ 1 \\ 1 \end{bmatrix}$$

© Copyright EE, NUS. All Rights Reserved.

Polynomial Regression: Example (Prediction)

Test set

Test point 1: $\{x_1 = 0.1,\ x_2 = 0.1\} \rightarrow \{y = \text{class } -1 \text{ or } +1?\}$
Test point 2: $\{x_1 = 0.9,\ x_2 = 0.9\} \rightarrow \{y = \text{class } -1 \text{ or } +1?\}$
Test point 3: $\{x_1 = 0.1,\ x_2 = 0.9\} \rightarrow \{y = \text{class } -1 \text{ or } +1?\}$
Test point 4: $\{x_1 = 0.9,\ x_2 = 0.1\} \rightarrow \{y = \text{class } -1 \text{ or } +1?\}$

Binary classification

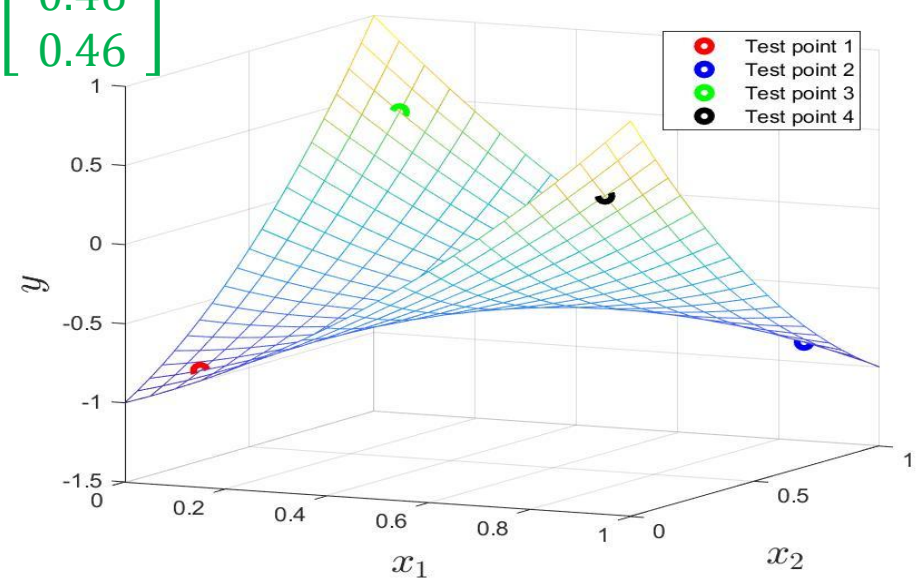
$$\hat{\mathbf{y}}_{raw} = \mathbf{P}_{new}\,\hat{\mathbf{w}}$$

$$= \begin{bmatrix} 1 & 0.1 & 0.1 & 0.01 & 0.01 & 0.01 \\ 1 & 0.9 & 0.9 & 0.81 & 0.81 & 0.81 \\ 1 & 0.1 & 0.9 & 0.09 & 0.01 & 0.81 \\ 1 & 0.9 & 0.1 & 0.09 & 0.81 & 0.01 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \\ 1 \\ -4 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.82 \\ -0.82 \\ 0.46 \\ 0.46 \end{bmatrix}$$

$$[bias \quad x_1 \quad x_2 \quad x_1x_2 \quad x_1^2 \quad x_2^2]$$

$$\hat{\mathbf{y}}_{new} = \text{sign}(\hat{\mathbf{y}}_{raw}) = \text{sign}\left(\begin{bmatrix} -0.82 \\ -0.82 \\ 0.46 \\ 0.46 \end{bmatrix}\right)$$

$$= \begin{bmatrix} -1 \\ -1 \\ +1 \\ +1 \end{bmatrix} \begin{array}{l} \dashrightarrow \text{Class } -1 \\ \dashrightarrow \text{Class } -1 \\ \dashrightarrow \text{Class } +1 \\ \dashrightarrow \text{Class } +1 \end{array}$$



Python demo

© Copyright EE, NUS. All Rights Reserved.

Polynomial Regression: Example (Python)

```python
#EE2211 Lecture 6 Demo 3 Binary Classification Using Polynomial regression
import numpy as np
from numpy.linalg import inv
from numpy.linalg import matrix_rank
from sklearn.preprocessing import PolynomialFeatures
X = np.array([ [0, 0], [1, 1], [1, 0], [0, 1]])
y = np.array([[-1], [-1], [1], [1]])

## Generate polynomial features
order = 2
poly = PolynomialFeatures(order)
P = poly.fit_transform(X)
print("matrix P")
print(P)
```

Transform **X** to **P** using PolynomialFeatures

PolynomialFeatures(order):

$X \rightarrow P$

- Generate a new feature matrix consisting of all polynomial combinations of the features with degree less than or equal to the specified degree.
- Bias column is automatically included.

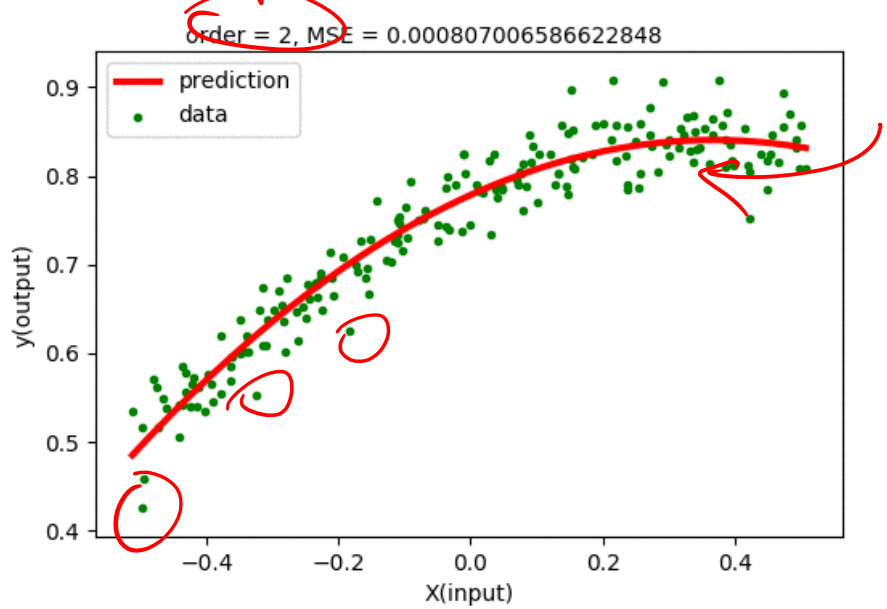
© Copyright EE, NUS. All Rights Reserved.

Polynomial Regression: Summary

- Polynomial regression increases the feature dimensionality by creating higher-order terms through feature multiplication.

- Higher dimensionality (higher-order) does not necessarily improve performance and can increase computational cost.
  - The number of terms (P/w size) grow exponentially
  - Computational cost will be increased
  - Prone to over-fitting (regularizations may be needed)

polynomial order



Source: https://aiinarabic.com/regression/

© Copyright EE, NUS. All Rights Reserved.

Quick Question 2

True or False: Suppose we would like to build up one full third-order polynomial model for only one input feature and single output, the polynomial model has three parameters to learn.

PollEv.com/shaojing

Related to TUT6-Q2 & Q7
Exam-style question

© Copyright EE, NUS. All Rights Reserved.

Quick Question 2

True or False: Suppose we would like to build up one full third-order polynomial model for only one input feature and single output, the polynomial model has three parameters to learn. ✗

$$y = w_0 + w_1 x + w_2 x^2 + w_3 x^3$$

- w_0 is the bias term (don't forget the bias!)
- $w_1 x$ is the linear term
- $w_2 x^2$ is the quadratic term
- $w_3 x^3$ is the cubic term

The number of parameters depends on the number of original input features and the degree of the polynomial expansion.

Total number of parameters (including bias) $= C(d+n, n) = \frac{(d+n)!}{n!d!}$

d: The number of original input features (excluding bias)
n: The polynomial degree

© Copyright EE, NUS. All Rights Reserved.

© Copyright EE, NUS. All Rights Reserved.

Agenda

- Linear Regression for Classification
- Polynomial Regression
- Ridge Regression (Linear/polynomial Regression + L2 Regularization)

Regularization helps tackle some of the issues we encounter in linear/polynomial regression.

Its usefulness goes beyond this — it is widely applied in nearly all ML models, including deep neural networks.

© Copyright EE, NUS. All Rights Reserved.

Ridge Regression: Motivation

Recall Linear regression $\mathbf{X}\mathbf{w} = \mathbf{y}$

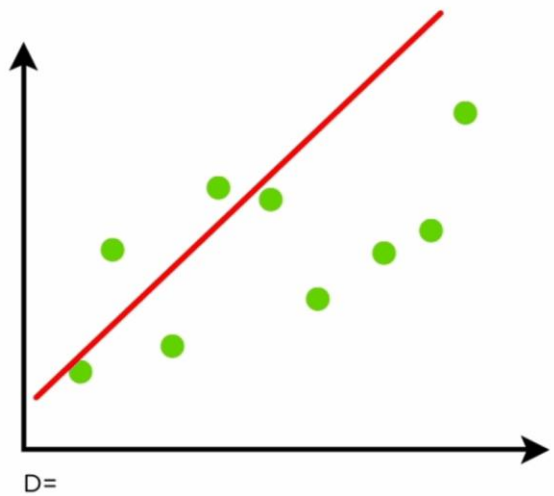
Objective: $\hat{\mathbf{w}} = \text{argmin} J(\mathbf{w}) = \text{argmin} \sum_{i=1}^{m} (f_{\mathbf{w}}(\mathbf{x}_i) - \mathrm{y}_i)^2$

$$= \text{argmin} \parallel \mathbf{X}\mathbf{w} - y \parallel^2 = \text{argmin} (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y})$$

If $\mathbf{X}^T\mathbf{X}$ is invertible, the learning computation: $\hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$

Problem: $\mathbf{X}^T\mathbf{X}$ is NOT always invertible

Solution: by adding a regularization term $(\lambda\mathbf{I})$, $\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}$ becomes **always invertible** for any $\lambda > 0$, ensuring we can always compute an approximate solution.



Linear regression finds the model parameters by ***minimizing a cost function*** that **measures prediction error.**

Source: https://medium.com/@novus_afk/linear-regression-an-overview-13d37a6bc4dd

© Copyright EE, NUS. All Rights Reserved.

Ridge Regression: Motivation

Linear regression $\mathbf{X}\mathbf{w} = \mathbf{y}$

Objective: $\hat{\mathbf{w}} = \text{argmin} J(\mathbf{w}) = \text{argmin} \sum_{i=1}^{m} (f_{\mathbf{w}}(\mathbf{x}_i) - \mathrm{y}_i)^2 = \text{argmin}\ (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y})$

The learning computation: $\hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$

Matrix $\mathbf{X}^T\mathbf{X}$ is NOT always invertible

Ridge regression solves this by adding a small positive term to the diagonal:

$$\hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\ \mathbf{X}^T\mathbf{y}$$

The $\lambda\mathbf{I}$ term guarantees the matrix $\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}$ is always invertible for $\lambda > 0$

Returning to the objective function, adding $\lambda\mathbf{I}$ in the <u>solution</u> is *equivalent to* adding a penalty term $\lambda \sum_{j=1}^{d} w_j^2$ to the <u>objective function</u>.

$w_1^2 + w_2^2 + \cdots + w_d^2$

Objective: $\hat{\mathbf{w}} = \text{argmin} \sum_{i=1}^{m} (f_{\mathbf{w}}(\mathbf{x}_i) - \mathrm{y}_i)^2 + \lambda \sum_{j=1}^{d} w_j^2$

$$= \text{argmin}\ (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}^T\mathbf{w}$$

Source: https://medium.com/@novus_afk/linear-regression-an-overview-13d37a6bc4dd

© Copyright EE, NUS. All Rights Reserved.

Ridge Regression: Motivation

$J(w) = \text{Error} + \text{Reg}$

Deduction:

Objective: $\hat{\mathbf{w}} = \text{argmin} \sum_{i=1}^{m} (f_{\mathbf{w}}(\mathbf{x}_i) - y_i)^2 + \lambda \sum_{j=1}^{d} w_j^2$

$= \text{argmin}\ (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}^T \mathbf{w}$

L_2

$\lambda \sum w_j^2$

Solution:

$$\frac{\partial}{\partial \mathbf{w}} \left((\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}^T \mathbf{w} \right) = \mathbf{0}$$

$$\Rightarrow 2\mathbf{X}^T\mathbf{X}\mathbf{w} - 2\mathbf{X}^T\mathbf{y} + 2\lambda\mathbf{w} = \mathbf{0}$$

$$\Rightarrow \mathbf{X}^T\mathbf{X}\mathbf{w} + \lambda\mathbf{w} = \mathbf{X}^T\mathbf{y}$$

$$\Rightarrow (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})\mathbf{w} = \mathbf{X}^T\mathbf{y}$$

$$\Rightarrow \hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y}$$

X^TX

always invertible for $\lambda > 0$

scalar

Source: https://medium.com/@novus_afk/linear-regression-an-overview-13d37a6bc4dd

© Copyright EE, NUS. All Rights Reserved.

Ridge Regression: Effect of L2 Regularization

Ridge regression (L2 regularization) adds a penalty term $\lambda \sum_{j=1}^{d} w_j^2$ to the objective function.

This penalty term:

- Ensures the optimization problem has a unique and stable solution, and
- Shrinks the regression coefficients by penalizing large weights.

Objective: $\hat{\mathbf{w}} = \text{argmin} \sum_{i=1}^{m} (f_{\mathbf{w}}(\mathbf{x}_i) - \mathrm{y}_i)^2 + \lambda \sum_{j=1}^{d} w_j^2$

$$= \text{argmin}\ (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}^T \mathbf{w}$$

Here $\lambda \geq 0$ is a regularization parameter that controls the amount of shrinkage: the larger the value of λ, the greater the amount of shrinkage.

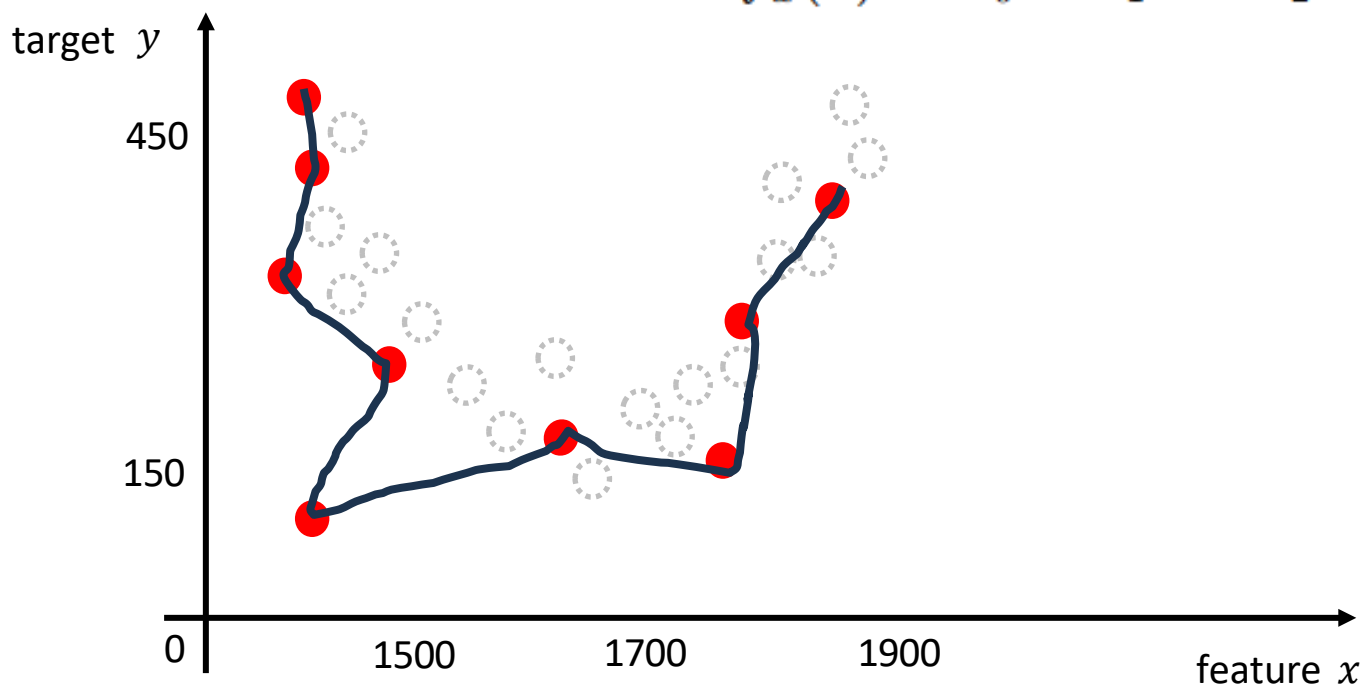
We are referring to the **magnitude** of w

© Copyright EE, NUS. All Rights Reserved.

Ridge Regression: Motivation (Another Perspective)

Red dots: training data

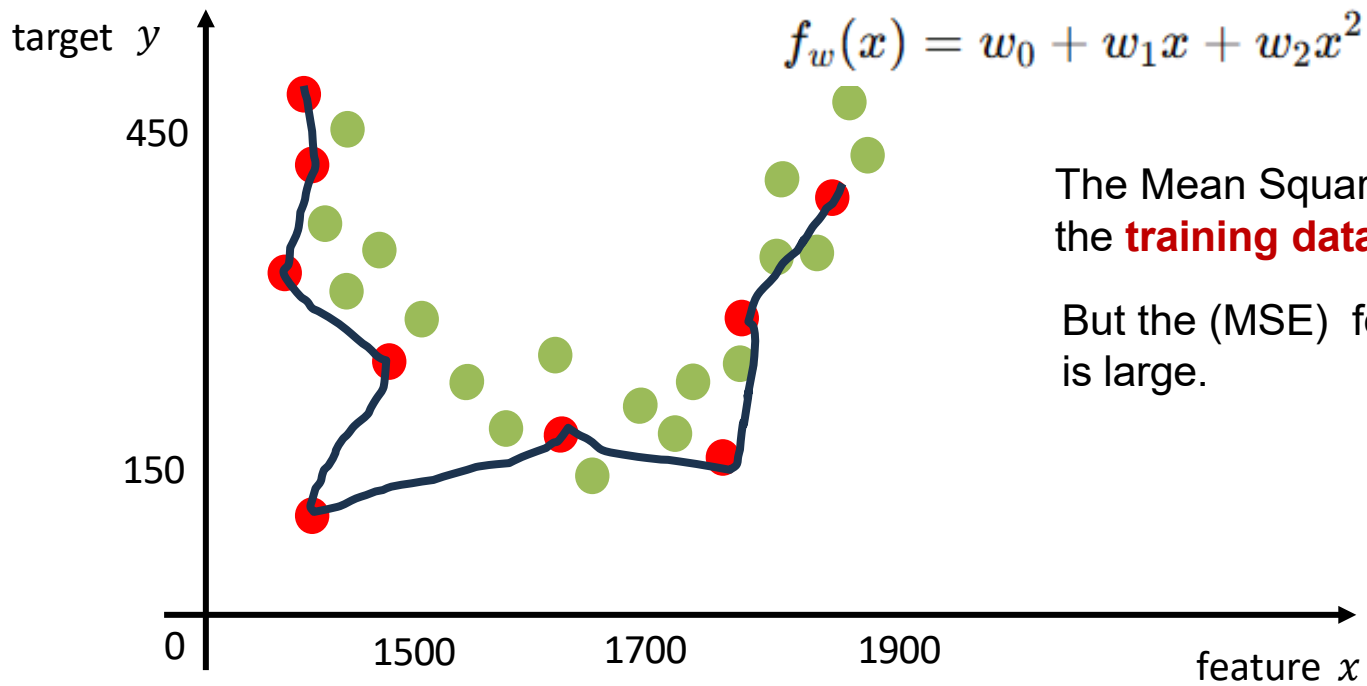
$$f_w(x) = w_0 + w_1 x + w_2 x^2 + w_3 x^3 + w_4 x^4$$



© Copyright EE, NUS. All Rights Reserved.

Ridge Regression: Motivation (Another Perspective)

Red dots: training data
Green dots: testing data



$$f_w(x) = w_0 + w_1 x + w_2 x^2 + w_3 x^3 + w_4 x^4$$

The Mean Squared Error (MSE) for the **training data** is small.

But the (MSE) for the **testing data** is large.

© Copyright EE, NUS. All Rights Reserved.

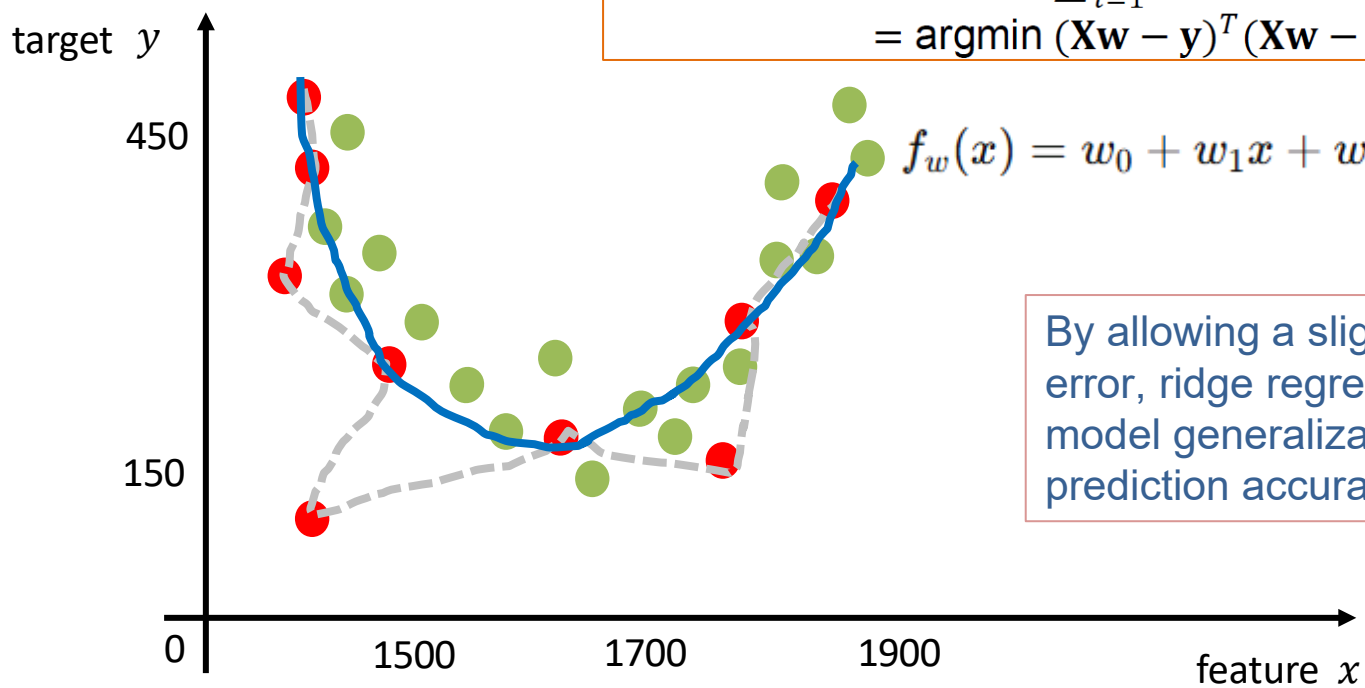
Ridge Regression: Motivation (Another Perspective)

Ridge regression: shrinks the regression coefficients *w* by imposing a penalty on their size

Error + Reg

Objective: $\widehat{\mathbf{w}} = \text{argmin} \sum_{i=1}^{m} (f_{\mathbf{w}}(\mathbf{x}_i) - \mathrm{y}_i)^2 + \lambda \sum_{j=1}^{d} w_j^2$

$= \text{argmin}\ (\mathbf{Xw} - \mathbf{y})^T (\mathbf{Xw} - \mathbf{y}) + \lambda \mathbf{w}^T \mathbf{w}$



$$f_w(x) = w_0 + w_1 x + w_2 x^2 + \underbrace{w_3 x^3}_{\approx 0} + \underbrace{w_4 x^4}_{\approx 0}$$

By allowing a slightly higher training error, ridge regression enhances model generalization and improves prediction accuracy on **unseen data.**

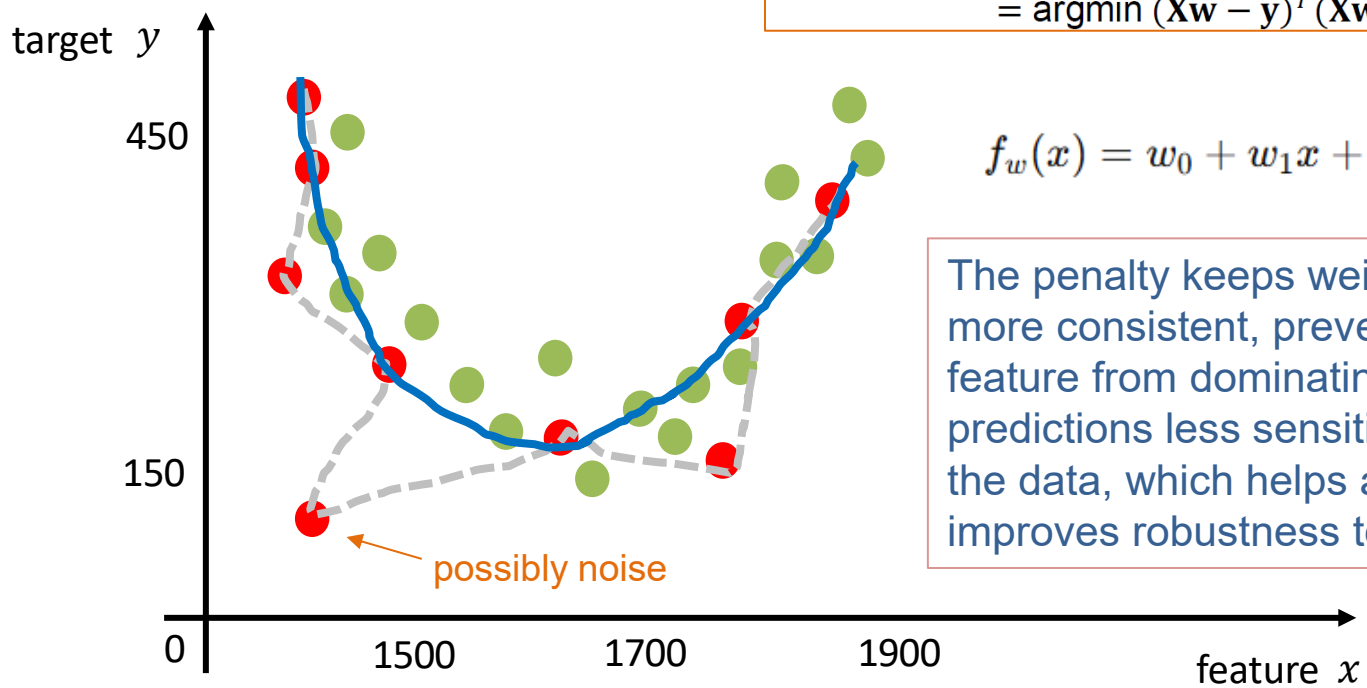
© Copyright EE, NUS. All Rights Reserved.

Ridge Regression: Motivation (Another Perspective)

Ridge regression: shrinks the regression coefficients *w* by imposing a penalty on their size

Objective: $\widehat{\mathbf{w}} = \text{argmin} \sum_{i=1}^{m} (f_{\mathbf{w}}(\mathbf{x}_i) - \mathrm{y}_i)^2 + \lambda \sum_{j=1}^{d} w_j^2$

$= \text{argmin}\ (\mathbf{Xw} - \mathbf{y})^T (\mathbf{Xw} - \mathbf{y}) + \lambda \mathbf{w}^T \mathbf{w}$



$$f_w(x) = w_0 + w_1 x + w_2 x^2 + \underbrace{w_3 x^3}_{\approx 0} + \underbrace{w_4 x^4}_{\approx 0}$$

The penalty keeps weights smaller and more consistent, preventing any single feature from dominating, and making predictions less sensitive to fluctuations in the data, which helps avoid overfitting and improves robustness to noise.

© Copyright EE, NUS. All Rights Reserved.

Ridge Regression: Summary

Ridge regression: shrinks the regression coefficients w by imposing a penalty on their size

Objective: $\widehat{\mathbf{w}} = \text{argmin} \sum_{i=1}^{m} (f_{\mathbf{w}}(\mathbf{x}_i) - \mathrm{y}_i)^2 + \lambda \sum_{j=1}^{d} w_j^2$

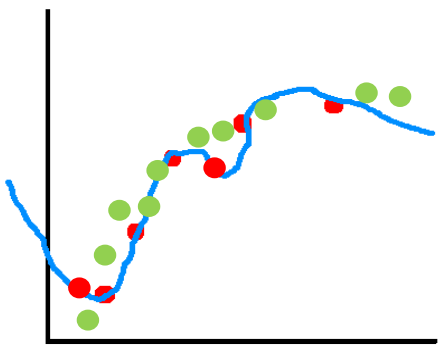
$= \text{argmin}\ (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}^T \mathbf{w}$

We are referring to the **magnitude** of w

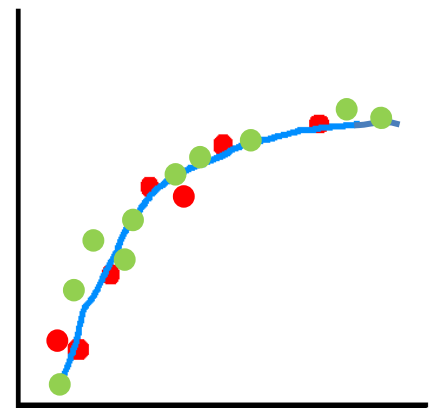
Smaller values for parameters w_i

-- "Simpler" hypothesis

-- Less prone to overfitting



$$y_{pred} = w_0 + w_1 x + w_2 x^2 + w_3 x^3 + w_4 x^4$$



$$y_{pred} = w_0 + w_1 x + w_2 x^2 + \cancel{w_3 x^3 + w_4 x^4}$$

Source: https://www.coryjmaklin.com/2019-08-19_Ridge-Regression-Python-Example-f015345d936b/

© Copyright EE, NUS. All Rights Reserved.

Ridge Regression: Application Example

Task: Predicting cancer risk based on gene expression

Inputs (Features, $\mathbf{x}$): Gene expression levels of 10,000 genes for each patient.
Output (Target, y): A binary variable indicating whether each patient developed cancer (1) or not (0).

$$y = w_0 + w_1 x_1 + w_2 x_2 + \ldots + w_{10000} x_{10000}$$

What do we do if we have an equation with 10,001 parameters and only 500 data points? (under-determined system)

Ans: We use ridge regression!

By introducing the penalty, ridge regression ensures that the model doesn't assign excessive weight to any single gene or data sample, leading to better generalization on unseen patient data.

Quick question: Why not least-norm solution?

© Copyright EE, NUS. All Rights Reserved.

Ridge Regression: Application Example

Least-norm:

exact

"Fit the training data perfectly, then pick the smallest weights."

- No explicit preference for any feature
- Can fit noise
- Does not remove unimportant genes

L2 regularization (error + $\lambda \mathbf{w}^T \mathbf{w}$):

$w \rightarrow$ useless $\rightarrow 0$

"Fit the data, but pay a cost for every weight you use."

- A feature must reduce error enough to justify its cost
- Weak/noisy features get shrunk
- Produces a simpler and more stable model

Key difference:

- **Least-norm:** Finds the smallest (minimum norm) solution among the infinitely many that perfectly fit the training data.
- **Regularization:** Finds a solution that approximately fits the training data while actively controlling model complexity (by keeping weights small) to improve generalization.

© Copyright EE, NUS. All Rights Reserved.

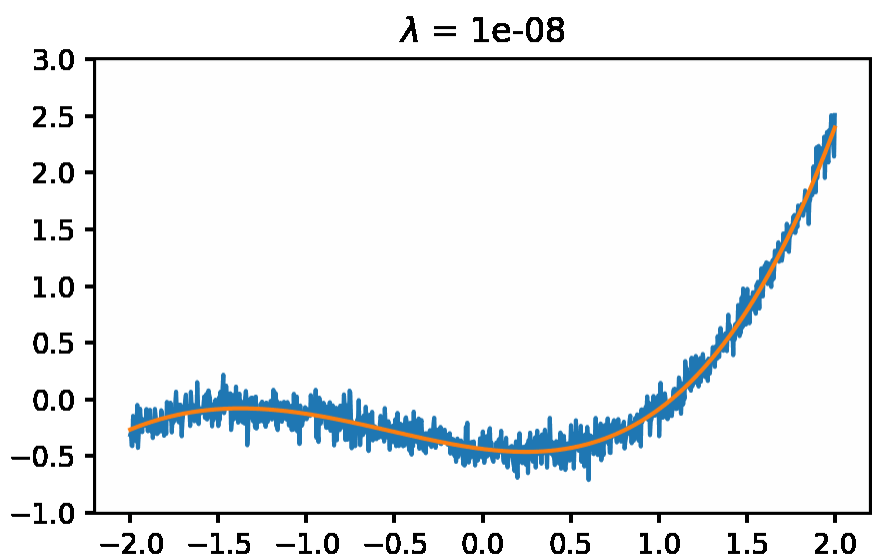
Ridge Regression: Practical Considerations

Ridge regression (L2 regularization): shrinks the regression coefficients $\mathbf{w}$ by imposing a penalty on their size

Error reg

Objective: $\hat{\mathbf{w}} = \text{argmin} \sum_{i=1}^{m} (f_{\mathbf{w}}(\mathbf{x}_i) - \mathrm{y}_i)^2 + \lambda \sum_{j=1}^{d} w_j^2$

$= \text{argmin}\ (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}^T \mathbf{w}$



In practice, λ controls the **trade-off** between two different goals:

i) fit the training data well;

ii) keep the parameters small

A larger λ favors a simpler model with smoother curves, while a smaller λ allows for more complexity and greater curvature. By adjusting λ, we can fine-tune this trade-off to achieve the best generalization performance.

Source: https://dkenefake.github.io/blog/Ridge_Regression

© Copyright EE, NUS. All Rights Reserved.

Ridge Regression: Learning

Objective: $\hat{\mathbf{w}} = \text{argmin} \sum_{i=1}^{m} (f_{\mathbf{w}}(\mathbf{x}_i) - \mathrm{y}_i)^2 + \lambda \sum_{j=1}^{d} w_j^2$

$$= \text{argmin}\ (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}^T \mathbf{w}$$

Solution:

$$\frac{\partial}{\partial \mathbf{w}} \left((\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}^T \mathbf{w} \right) = \mathbf{0}$$

$$\Rightarrow 2\mathbf{X}^T\mathbf{X}\mathbf{w} - 2\mathbf{X}^T\mathbf{y} + 2\lambda\mathbf{w} = \mathbf{0}$$

$$\Rightarrow \mathbf{X}^T\mathbf{X}\mathbf{w} + \lambda\mathbf{w} = \mathbf{X}^T\mathbf{y}$$

$$\Rightarrow (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})\mathbf{w} = \mathbf{X}^T\mathbf{y}$$

$$\Rightarrow \hat{\mathbf{w}} = \underline{(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})}^{-1}\, \mathbf{X}^T\mathbf{y}$$

always invertible for $\lambda > 0$

Learning: $\hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\, \mathbf{X}^T\mathbf{y}$

Quick question: What is the dimension of matrix $\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}$?

m: no. of samples, d: no. of feature dimensions

Source: https://medium.com/@novus_afk/linear-regression-an-overview-13d37a6bc4dd

© Copyright EE, NUS. All Rights Reserved.

Ridge Regression: Learning

Objective: $\hat{\mathbf{w}} = \text{argmin} \sum_{i=1}^{m} (f_{\mathbf{w}}(\mathbf{x}_i) - \mathrm{y}_i)^2 + \lambda \sum_{j=1}^{d} w_j^2$

$$= \text{argmin}\ (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}^T \mathbf{w}$$

Solution:

$$\frac{\partial}{\partial \mathbf{w}} \left((\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}^T \mathbf{w} \right) = \mathbf{0}$$

$$\Rightarrow 2\mathbf{X}^T\mathbf{X}\mathbf{w} - 2\mathbf{X}^T\mathbf{y} + 2\lambda\mathbf{w} = \mathbf{0}$$

$$\Rightarrow \mathbf{X}^T\mathbf{X}\mathbf{w} + \lambda\mathbf{w} = \mathbf{X}^T\mathbf{y}$$

$$\Rightarrow (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})\mathbf{w} = \mathbf{X}^T\mathbf{y}$$

$$\Rightarrow \hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\, \mathbf{X}^T\mathbf{y}$$

always invertible for $\lambda > 0$

m: no. of samples,
d: no. of feature dimensions

Learning: $\hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\, \mathbf{X}^T\mathbf{y}$

$\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}$ is a $d \times d$ matrix. In under-determined systems ($m < d$), d could be very large. Computing $(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}$ will be computationally expensive.

Source: https://medium.com/@novus_afk/linear-regression-an-overview-13d37a6bc4dd

© Copyright EE, NUS. All Rights Reserved.

Ridge Regression: Learning

Related to TUT6-Q1

Claim*: For any $(\mathbf{X}, \mathbf{y})$, if $\lambda > 0$,

$$\hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\,\mathbf{X}^T\mathbf{y} \quad \leftarrow \textbf{Primal Form}$$

$d \times d$

$$= \mathbf{X}^T(\mathbf{X}\mathbf{X}^T + \lambda\mathbf{I})^{-1}\,\mathbf{y} \quad \leftarrow \textbf{Dual Form}$$

$m \times m$

Rationale for Primal and Dual form:

- $\mathbf{X}^\mathbf{T}\mathbf{X} + \lambda\mathbf{I}$ is a $d \times d$ matrix. In under-determined systems $(m < d)$, d can be very large, making the computation of $(\mathbf{X}^\mathbf{T}\mathbf{X} + \lambda\mathbf{I})^{-1}$ in the primal form computationally expensive.
- In the dual form, we instead invert an $m \times m$ matrix, which is more efficient when $m \ll d$.
- Conceptually:
  - **Primal form** is analogous to the **left-inverse** in over-determined systems.
  - **Dual form** is analogous to the **right-inverse** in under-determined systems.

*Refer to the proof is in Tutorial Q1.

© Copyright EE, NUS. All Rights Reserved.

Ridge Regression: Learning & Prediction

Ridge Regression in Primal Form (preferred when $m > d$)

$(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})$ is invertible for $\lambda > 0$,

Learning: $\hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\,\mathbf{X}^T\mathbf{y}$

Prediction: $\hat{f}_{\mathbf{w}}(\mathbf{X}_{new}) = \mathbf{X}_{new}\hat{\mathbf{w}}$

Ridge Regression in Dual Form (preferred when $m < d$)

$(\mathbf{X}\mathbf{X}^T + \lambda\mathbf{I})$ is invertible for $\lambda > 0$,

Learning: $\hat{\mathbf{w}} = \mathbf{X}^T(\mathbf{X}\mathbf{X}^T + \lambda\mathbf{I})^{-1}\,\mathbf{y}$

Prediction: $\hat{f}_{\mathbf{w}}(\mathbf{X}_{new}) = \mathbf{X}_{new}\hat{\mathbf{w}}$

*The primal and dual forms are two **equivalent ways** to learn regression models, with different computational costs depending on the number of features and samples.*

Ref: Hastie, Tibshirani, Friedman, "The Elements of Statistical Learning", (2nd ed., 12th printing) 2017 (chp.3)

© Copyright EE, NUS. All Rights Reserved.

Polynomial Regression with Ridge

Ridge Regression in Primal Form (preferred when $m > d$)

$(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})$ is invertible for $\lambda > 0$,

Learning: $\hat{\mathbf{w}} = (\mathbf{P}^T\mathbf{P} + \lambda\mathbf{I})^{-1}\,\mathbf{P}^T\mathbf{y}$

Prediction: $\hat{f}_{\mathbf{w}}(\mathbf{P}_{new}) = \mathbf{P}_{new}\hat{\mathbf{w}}$

Ridge Regression in Dual Form (preferred when $m < d$)

$(\mathbf{X}\mathbf{X}^T + \lambda\mathbf{I})$ is invertible for $\lambda > 0$,

Learning: $\hat{\mathbf{w}} = \mathbf{P}^T(\mathbf{P}\mathbf{P}^T + \lambda\mathbf{I})^{-1}\,\mathbf{y}$

Prediction: $\hat{f}_{\mathbf{w}}(\mathbf{P}_{new}) = \mathbf{P}_{new}\hat{\mathbf{w}}$

Note: m & d refer to the size of $\mathbf{P}$ (not $\mathbf{X}$)

© Copyright EE, NUS. All Rights Reserved.

Summary

Aspect	Primal Form	Dual Form
Learning solution	$\hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\,\mathbf{X}^T\mathbf{y}$	$\hat{\mathbf{w}} = \mathbf{X}^T(\mathbf{X}\mathbf{X}^T + \lambda\mathbf{I})^{-1}\,\mathbf{y}$
Matrix to invert	$d \times d$ (# of features)	$m \times m$ (# of samples)
Preferred when	$d \ll m$ (few features, many samples)	$d \gg m$ (many features, few samples)
Computational cost	Low if d is small	Low if m is small
Notes	Simple and direct	Useful for very high-dimensional features
Example	Polynomial regression with 2 features and 10 samples (d=6, m=10)	Gene expression data with 10,000 genes and 500 samples (d=10,000, m=500)

Notes: for polynomial regression, transform $\mathbf{X}$ to $\mathbf{P}$, and m & d refer to the size of $\mathbf{P}$ (not $\mathbf{X}$)
In this course, we use the terms primal form and dual form even when there is no regularization.

© Copyright EE, NUS. All Rights Reserved.

Summary (cont.)

- Binary and multi-category classification using linear/polynomial regression
- Polynomial Regression (transform $\mathbf{X}$ to $\mathbf{P}$)
- Ridge Regression (L2 regularization): penalty term, primal and dual forms

* Most programming-related questions will be based on today's content.
* Useful Python packages and functions
sklearn.preprocessing (PolynomialFeatures), np.sign, sklearn.model_selection (train_test_split), sklearn.preprocessing (OneHotEncoder)

© Copyright EE, NUS. All Rights Reserved.

References

Regularization to reduce overfitting by Dr. Andrew Ng:
https://www.youtube.com/watch?v=8upNQi-40Q8&list=PLkDaE6sCZn6FNC6YRfRQc_FbeQrF8BwGI&index=37

https://www.youtube.com/watch?v=1kgcON0Eauc&list=PLkDaE6sCZn6FNC6YRfRQc_FbeQrF8BwGI&index=38

https://www.youtube.com/watch?v=NIiZZY7nlfU&list=PLkDaE6sCZn6FNC6YRfRQc_FbeQrF8BwGI&index=39

https://www.youtube.com/watch?v=jhrrw8Iuus0&list=PLkDaE6sCZn6FNC6YRfRQc_FbeQrF8BwGI&index=40

Regularization and different types:
https://www.ibm.com/think/topics/regularization

Polynomial regression by Dr. Andrew Ng:
https://www.youtube.com/watch?v=IFkRKJ5iBDE&list=PLkDaE6sCZn6FNC6YRfRQc_FbeQrF8BwGI&index=30

© Copyright EE, NUS. All Rights Reserved.

Announcement

Assignments

- A1 (5%) is due today (20 Feb 23:59pm)
- A2 (15%) will be released later today. **GAs in charge of A2**:
  - Yan Weilong (yanweilong@u.nus.edu)
  - Guo Jingyuan (e0724521@u.nus.edu)
- If you have any questions about A2, feel free to post them *anonymously* on the Canvas discussion board. Our GAs will respond as soon as they can.

Mid-term Exam (Saturday, 14 March 2026, 4–5 pm)

- Trial quiz released on Examplify (refer to announcement for details).
- We will share additional exam details and last semester's midterm paper for practice later.

Additional Python Clinic Session: Friday, Week 7 (6 March) 8-9pm

Additional office hour during Recess Week (Friday, 27 Feb, 10.30-11.30am)

- I will hold an additional office hour during **recess week** to help you better prepare for the midterm exams. This office hour will be conducted **fully online**.
- Regular office hours will return to hybrid mode from Week 7 onward.

© Copyright EE, NUS. All Rights Reserved.

© Copyright National University of Singapore. All Rights Reserved.